INDEX

INTRODUCTION

S.No.	Name of Experiment	Date of Allotment	Date of Evaluation	Signature

UML (Unified Modeling Language) is a standardized visual language used to represent the design and behavior of software systems. It is an essential tool in software engineering for system analysis, design, and documentation. UML diagrams are categorized into two types: structural and behavioral.

Structural UML Diagrams: These diagrams describe the static aspects of a system, like its architecture and relationships between components.

- Class Diagram: Shows the structure of classes, their attributes, methods, and relationships.
- Object Diagram: Represents instances of classes at a specific moment in time.
- Component Diagram: Illustrates system components (e.g., libraries, executables) and their dependencies.
- Deployment Diagram: Displays the physical distribution of software and hardware.
- Composite Structure Diagram: Shows internal structures and interactions within a class.
- Package Diagram: Organizes system components into packages and shows dependencies.

Behavioral UML Diagram: These diagrams focus on the dynamic aspects of a system, such as interactions and processes.

- Use Case Diagram: Depicts system functionality and interactions with external actors (users, other systems).
- Sequence Diagram: Visualizes the sequence of interactions between objects over time.
- Activity Diagrgram: Models workflows, including activities, conditions, and parallel processes.
- State Machine Diagram: Shows object states and transitions triggered by events.
- Communication Diagram: Highlights interactions between objects, focusing on message flow.
- Timing Diagram: Emphasizes time constraints and changes in object states.

Key UML Notations:

- Classes: Represented as rectangles with three sections for name, attributes, and methods.
- Actors: Stick figures in use case diagrams, representing external entities.
- Relationships:
- Association: Solid lines connecting elements.
- Dependency: Dotted arrows showing one element’s dependence on another.
- Generalization: Hollow arrowheads indicating inheritance.
- Composition: Filled diamonds representing strong "whole-part" relationships.

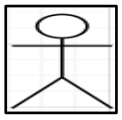
UML Tools: Tools like Lucidchart, Draw.io, Visual Paradigm, and StarUML help in creating UML diagrams, providing an intuitive interface for designing systems.

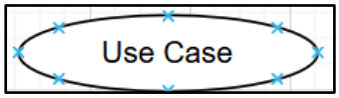
LAB Assignment 1

- **<u>Theory:</u>** A Use Case Diagram (UCD) is a type of behavioral diagram in UML that illustrates the interactions between actors (users or other systems) and the system under development. It represents the system's functional requirements from the user's perspective and describes how different actors interact with the system.
  - Actors: Represent external entities (such as users, systems, or hardware) that interact with the system. Actors initiate or participate in use cases.
  - Use Cases: Represent specific functions or tasks that the system can perform in response to an actor's action.
  - System: The boundary that separates the system from external actors and specifies the scope of the system.

  Use case diagrams help in understanding the functional requirements of a system and form the foundation for further system design and development.

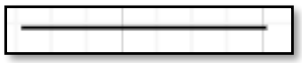
  Symbols in a Use Case Diagram:
  - Actor: Represented by a stick figure, actors can be humans, external systems, or other entities that interact with the system.

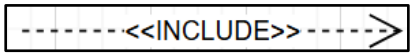
    
  - Use Case: Represented by an oval or ellipse, a use case defines a specific function or goal the system is meant to achieve.

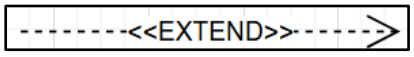
  - System Boundary: Represented by a rectangle, it defines the scope of the system. The use cases are enclosed within this boundary, and the actors interact with them outside this boundary.

  - Associations: Represented by a line, associations indicate the communication between actors and use cases, showing which actor participates in which use case.

    
  - Include: Represented by a dashed line with an open arrow, it shows that the functionality of one use case is always included in another use case.

  - Extend: Represented by a dashed line with an arrow, it indicates that one use case can optionally extend the behavior of another use case under certain conditions.

1(a)

- **Aim:** Use Case Diagram (UCD) of ATM System.

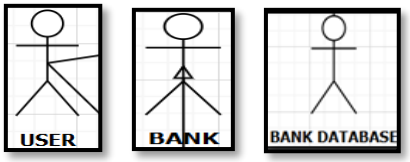
  The ATM Use Case Diagram represents the interaction between a User, the ATM System, and the Bank. It shows the various services provided by an ATM such as withdrawing cash, depositing cash, checking balance, and printing receipts. The diagram helps in understanding the functional requirements of the ATM system from the user's point of view and how the ATM interacts with the bank and database.

  Actors:
  - User: Represents the customer who uses the ATM to perform banking operations.
  - Bank: Represents the bank server that authenticates the user and maintains account details.
  - Bank Database: Stores user account details, balance information, and transaction records.

  Use Cases:
  - Insert Card: Allows the user to insert the ATM card to initiate a session.
  - Authenticate User: Verifies the user's credentials using ATM card details and PIN.
  - Perform Transaction: Represents the overall transaction process selected by the user.
  - Withdraw Cash: Enables the user to withdraw money from their account.
  - Deposit Cash: Allows the user to deposit money into their account.
  - Transfer Cash: Enables fund transfer between accounts.
  - Check Balance / View Balance: Displays the available account balance to the user.
  - Print Receipt: Prints transaction details after completion.
  - Maintenance: Performed by the bank to maintain and update the ATM system.

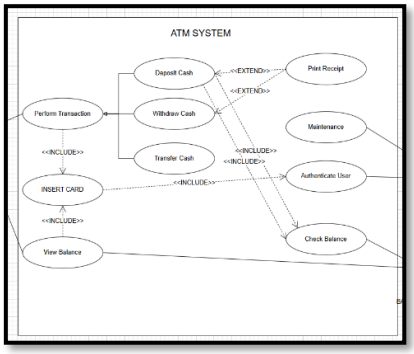
  Relationships:
  - Association: (Direct interaction between an actor and a use case.)
    - User is associated with Perform Transaction and Check Balance.
    - Bank is associated with Authenticate User and Maintenance.
    - Bank Database is associated with Check Balance and transaction processing.
  - Include (<<include>>): (Authentication is mandatory before performing any transaction.)
    - Perform Transaction includes Insert Card.
    - Insert Card includes Authenticate User.
    - Withdraw Cash, Deposit Cash, and Transfer Cash include Authenticate User.
    - View Balance includes Authenticate User.
  - Extend (<<extend>>): (Indicates optional or conditional functionality.)
    - Print Receipt extends Withdraw Cash and Deposit Cash (optional action).
  - Generalization : Withdraw Cash, Deposit Cash, and Transfer Cash are generalized from Perform Transaction. This means Perform Transaction is a common parent use case, and the others are specialized forms of transactions.

  Symbols Used:
  - Actor: Stick figure (User, Bank, Bank Database)

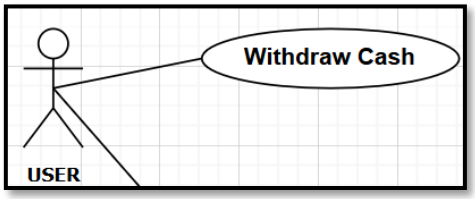


  - Use Case: Oval (Withdraw Cash, Deposit Cash, etc.)

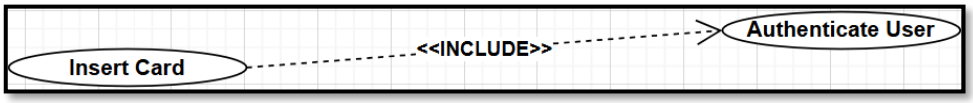
- System Boundary: Rectangle labeled ATM System



- Association: Solid line

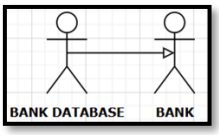


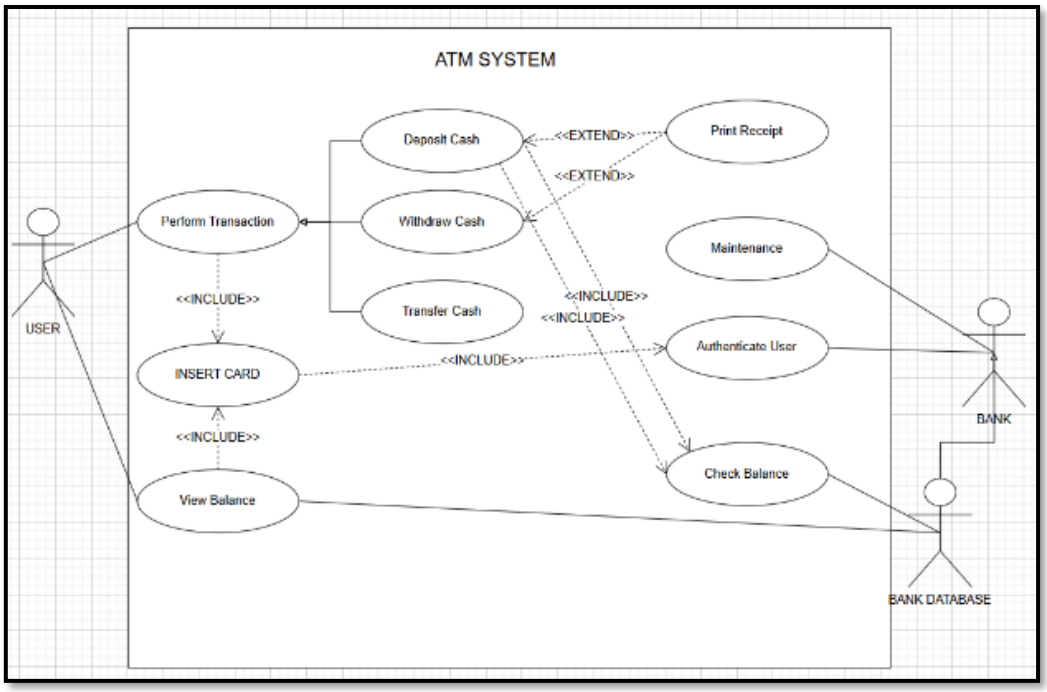
- Include: Dashed arrow with <<include>>



- Extend: Dashed arrow with <<extend>>

- Generalization: Solid line with open triangle arrow pointing to the actor or parent use case.





1(b)

- **Aim:** Use Case Diagram (UCD) of Amizone System.
  The Amizone Use Case Diagram represents a University Management System that manages academic, administrative, hostel, and placement-related activities. It shows how different users such as students, faculty, hostel wardens, placement cell, and administrators interact with the Amizone system. The diagram helps in understanding the functional flow of academic and non-academic services provided by the university.

  Actors:
  - Student: Represents university students who use the system for academic and administrative activities.
  - Hosteller / Day Scholar: Student classification based on accommodation.
  - Faculty (Faculty / HOD / PL): Manages academic activities.
  - Account Department: Manages fee-related activities.
  - IT Support / Authentication Server: Responsible for authentication and validation of users.
  - Placement Cell: Handles internships, placements, and corporate events.
  - Hostel Warden: Manages hostel-related activities.
  - University Server: Stores and manages all academic and administrative data.

  Use Cases:
  - Student Login: Allows student access to the system.
  - Authentication and Validation: Verifies credentials.
  - Register: Course/semester registration.
  - View Attendance: Displays attendance records.
  - View Marks: Displays academic results.
  - Fill APW: Academic Project Work submission.
  - Pay Fees / Hostel Fees: Fee payment services.
  - Schedule Class: Faculty schedules classes.
  - Approve APW: Faculty approval process.
  - Upload Attendance: Faculty uploads attendance.
  - Upload Internal Marks: Faculty uploads marks.
  - OD (On Duty): Allows students to mark attendance for official academic activities.
  - LMS (Learning Management System): Manages course-related learning content.
  - Upload Course Material: Faculty uploads study content.
  - View / Study Course Material: Students access materials.
  - Apply for Out Pass: Hostel permission process.
  - Hostel Accommodation: Hostel management.
  - Apply for Internship: Internship applications.
  - Apply for Placement: Placement applications.
  - Upload Resume: Resume submission.
  - Corporate Event: Placement-related events.
  - Stores and Manages All Activities: Central system management.

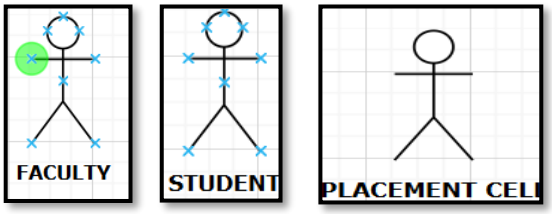
  Relationships
  - Association: (Direct interaction between an actor and a use case.)
    - Student is associated with Login, Register, View Attendance, View Marks, and Pay Fees.
    - Faculty is associated with Schedule Class, Upload Attendance, Upload Internal Marks, and Approve APW.
    - Placement Cell is associated with placement-related use cases.

- Hostel Warden is associated with hostel-related use cases.
- IT Support is associated with Authentication and Validation.

- Include (<<include>>): (Use case always requires another use case to be executed as part of its functionality.)
  - Register, View Attendance, View Marks, and Pay Fees include Student Login.
  - Approve includes Mark processing.
- Extend (<<extend>>): (indicates optional or conditional functionality.)
  - Reject extends Mark.
  - Upload Course Material and View Study Material extend LMS.
  - Apply for Internship, Apply for Placement, and Upload Resume extend ATPC/CRC.
- Generalization (Open Arrow): (Inheritance relationship where a child actor/use case is a specialized form of a parent.)
  - Hosteller and Day Scholar are specialized actors generalized from Student.
  - Faculty, HOD, and PL are specialized roles generalized from Faculty.
  - Pay Fees and Hostel Fees are specialized forms of the fee payment process.

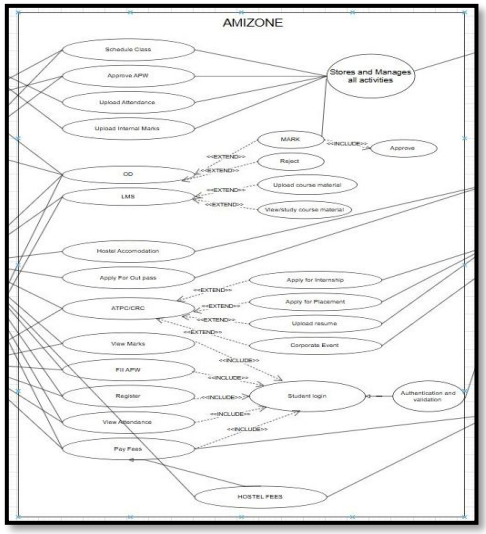
Symbols Used

- Actor: Stick figures (Student, Faculty, Admin, etc.)

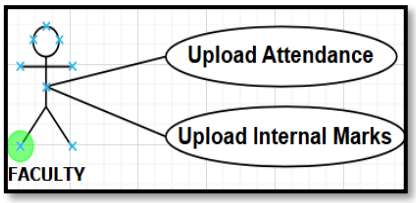


- Use Case: Ovals representing system functions

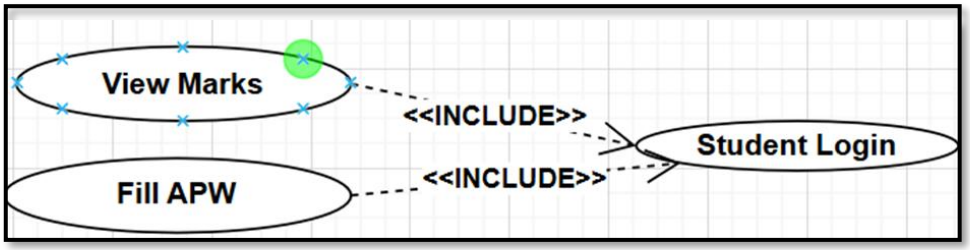
- System Boundary: Rectangle labeled AMIZONE



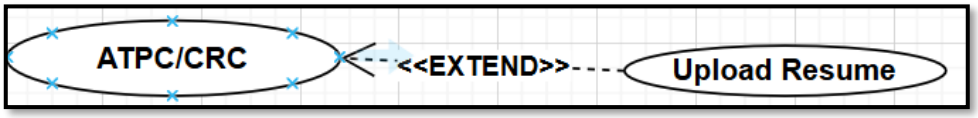
- Association: Solid line between actor and use case



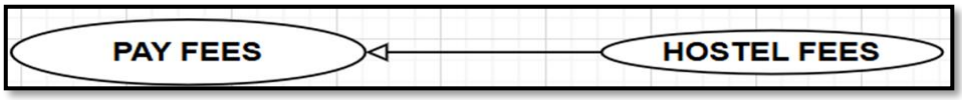
- Include (<<include>>): Dashed arrow indicating compulsory behavior

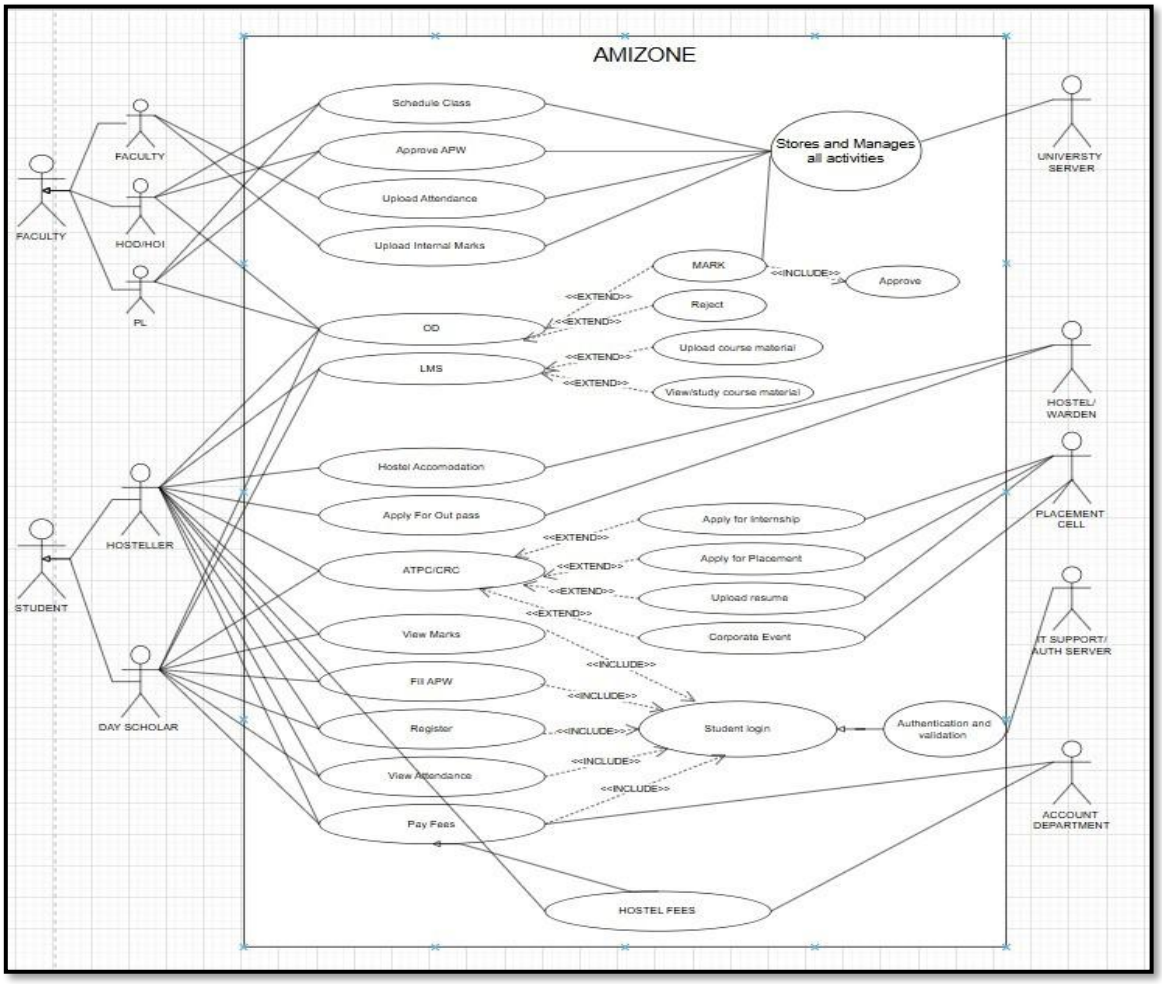


- Extend (<<extend>>): Dashed arrow indicating optional behavior



- Generalization: Solid line with open triangle arrow pointing toward the generalized actor or use case



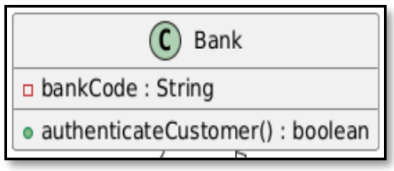


LAB Assignment 2

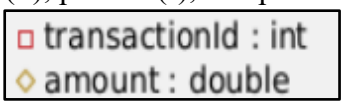
- **Theory:** A Class Diagram is a type of structural diagram in UML that represents the static structure of a system. It shows the system's classes, their attributes, methods (operations), and the relationships among classes. Class diagrams are used to model the design of a system and serve as a blueprint for system implementation.
  - Class: Represents a blueprint or template for creating objects. It defines the properties (attributes) and behaviors (methods) that the objects of the class will have.
  - Attributes: Represent the data or properties of a class. They describe the information that an object of the class can store.
  - Methods (Operations): Represent the functions or behaviors that a class can perform. They define how the class interacts with data or other classes.

  Class diagrams help in understanding the system's structure, organizing code, and visualizing relationships between different components of the system.

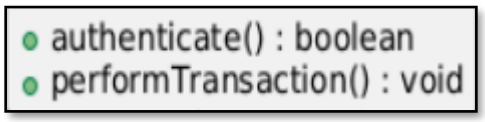
  Components of Class Diagram:
  - Class: Represented by a rectangle divided into three compartments showing the class name, attributes, and methods.



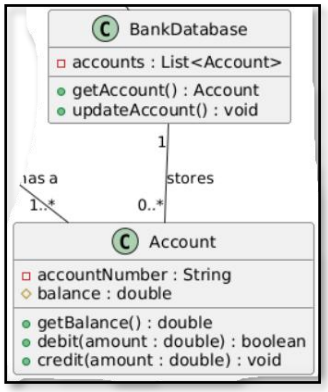
  - Attributes: Data members of a class written in the middle compartment, usually with visibility symbols like public (+), private (-), and protected (#).



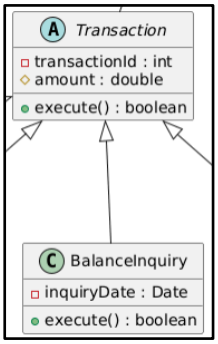
  - Methods (Operations): Functions of a class written in the bottom compartment, including visibility, parameters, and return types.



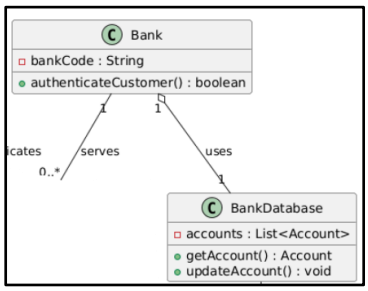
  - Association: A solid line connecting classes that represents a relationship or interaction between them.



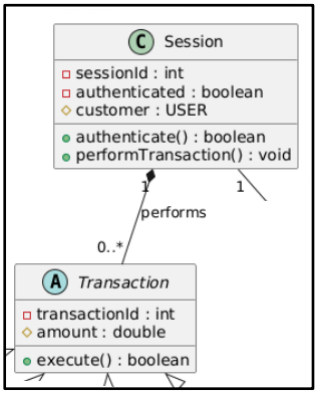
- Generalization: A solid line with a hollow triangle arrow showing that a subclass inherits attributes and methods from a superclass.



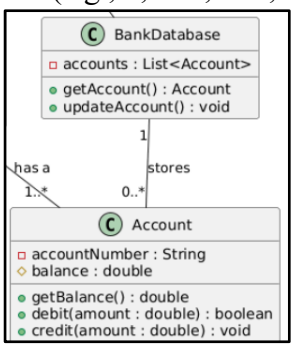
- Aggregation: A relationship shown by a hollow diamond indicating a “has-a” relationship where parts can exist independently.



- Composition: A relationship shown by a filled diamond indicating a strong “has-a” relationship where parts cannot exist independently.



- Multiplicity: Specifies the number of objects that can participate in a relationship, shown at the ends of an association (e.g., 1, 0..1, 1..*, *).



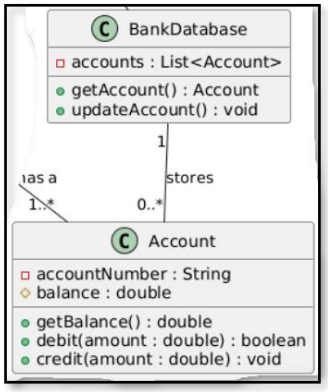
2(a)

- **Aim:** Class Diagram of ATM System.

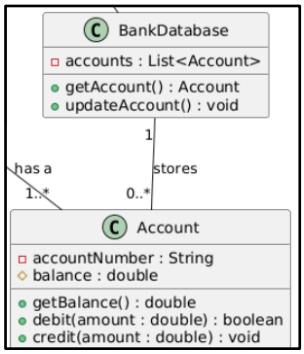
  The ATM Class Diagram represents the static structure of the ATM system by showing its classes, attributes, methods, and the relationships among them. It explains how different components of the ATM system are organized and how they interact with each other to support banking operations such as authentication and transactions.

  Classes:
  - Session: Manages user authentication and controls the execution of ATM transactions.
  - USER: Represents the customer and stores personal details and PIN for validation.
  - Bank: Authenticates users and coordinates banking services.
  - BankDatabase: Stores and manages account information and updates.
  - Account: Represents a bank account and supports balance, debit, and credit operations.
  - Transaction: An abstract class defining common properties and behavior for all transactions.
  - Withdraw: Handles cash withdrawal with amount and withdrawal limit.
  - Deposit: Handles cash deposit with deposit amount and date.
  - Transfer: Manages fund transfer between accounts.
  - BalanceInquiry: Allows the user to check account balance.

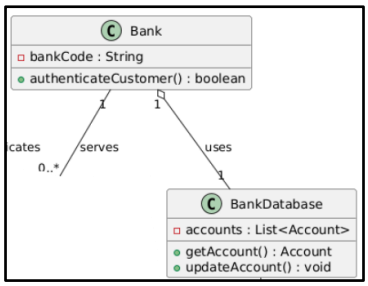
  Components:
  - Association: Represents a direct connection between two classes showing interaction, such as a Session authenticating a USER, a Bank serving multiple users, and a BankDatabase storing multiple accounts.



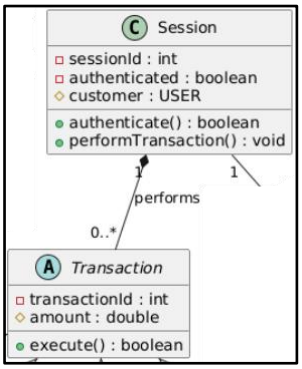
  - Multiplicity: Specifies the number of objects involved in a relationship, for example one Session is linked to one USER, one USER can have one or more Accounts, and one Session can perform zero or more Transactions.



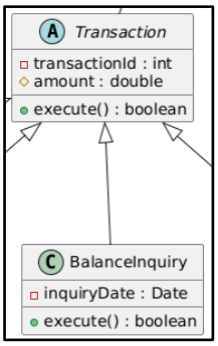
  - Aggregation: Represents a weak “has-a” relationship where objects can exist independently, such as a USER having multiple Accounts and a Bank using a BankDatabase.



- Composition: Represents a strong "has-a" relationship where the child object cannot exist without the parent, such as Transactions being part of a Session.



- Generalization: Represents an "is-a" relationship where subclasses inherit properties and methods from a parent class, such as Withdraw, Deposit, Transfer, and BalanceInquiry inheriting from the Transaction class.

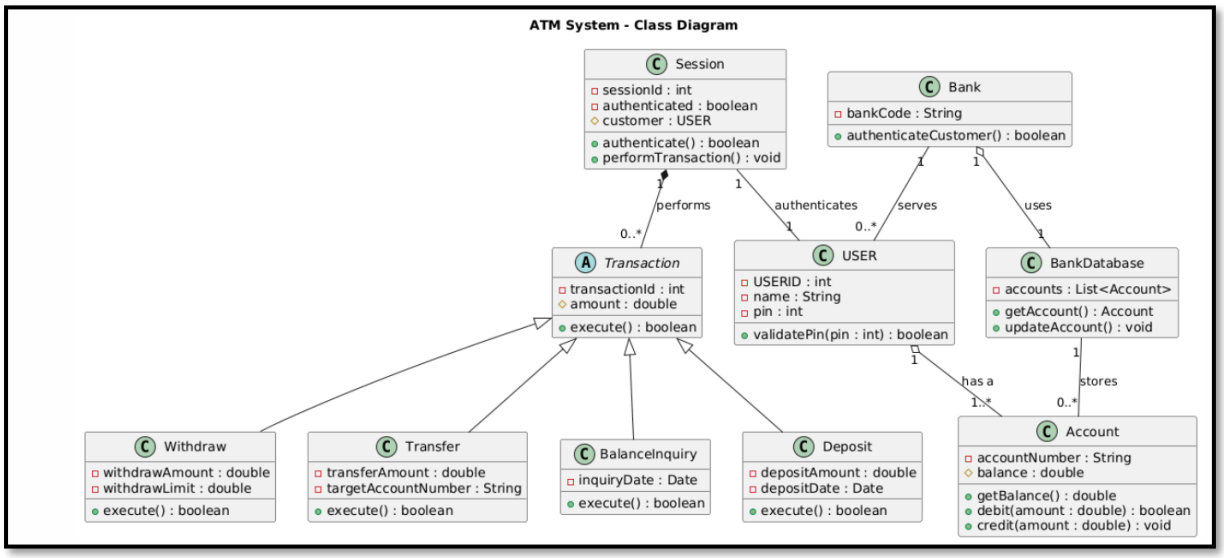


- **<u>CODE:</u>**

```
@startuml
title ATM System - Class Diagram
' Core Classes
class Session {
  - sessionId : int
  - authenticated : boolean
  # customer : USER
  + authenticate() : boolean
  + performTransaction() : void
}
class USER {
  - USERID : int
  - name : String
  - pin : int
  + validatePin(pin : int) : boolean
```

```
}

' Association with multiplicity
Session "1" -- "1" USER : authenticates

' Account Hierarchy
class Account {
  - accountNumber : String
  # balance : double
  + getBalance() : double
  + debit(amount : double) : boolean
  + credit(amount : double) : void
}

' Aggregation: USER owns accounts
USER "1" o-- "1..*" Account : has a

' Transaction Hierarchy
abstract class Transaction {
  - transactionId : int
  # amount : double
  + execute() : boolean
}

class Deposit {
  - depositAmount : double
  - depositDate : Date
  + execute() : boolean
}

class Withdraw {
  - withdrawAmount : double
  - withdrawLimit : double
  + execute() : boolean
}

class Transfer {
  - transferAmount : double
  - targetAccountNumber : String
  + execute() : boolean
}

class BalanceInquiry {
  - inquiryDate : Date
  + execute() : boolean
}
```

```
Transaction <|-- Deposit
Transaction <|-- Withdraw
Transaction <|-- Transfer
Transaction <|-- BalanceInquiry

' Composition: Session contains Transactions
Session "1" *-- "0..*" Transaction : performs

' Bank & Database
class Bank {
  - bankCode : String
  + authenticateCustomer() : boolean
}

class BankDatabase {
  - accounts : List<Account>
  + getAccount() : Account
  + updateAccount() : void
}

' Aggregation: Bank uses BankDatabase
Bank "1" o-- "1" BankDatabase : uses
' Associations with multiplicity
Bank "1" -- "0..*" USER : serves
BankDatabase "1" -- "0..*" Account : stores
@enduml
```



2(b)

- **Aim:** Class Diagram of Amizone System.

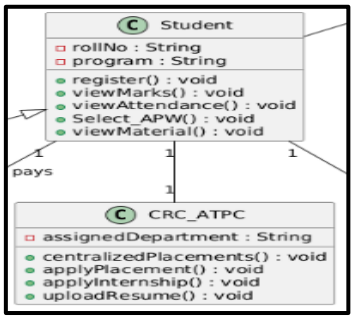
  The AMIZONE System Class Diagram represents the structure of a college management system website, showing the classes involved, their attributes, methods, and relationships. It helps in understanding how different user roles interact with the system to manage academic and administrative activities.

  Classes:

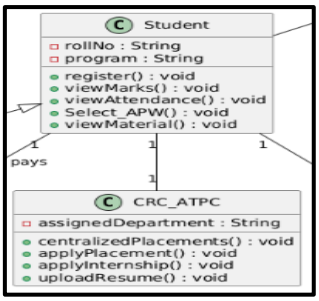
  - UniversityServer: Manages the database and system activities including storing data and managing operations.
  - User: Abstract class representing a general user with login and logout capabilities; inherited by specific user types.
  - Student: Represents students with attributes like roll number and program, and actions such as registration, viewing marks and attendance, and selecting activities.
  - Faculty: Represents faculty members who can upload attendance, internal marks, and course materials.
  - AuthServer: Responsible for authenticating users during the login process.
  - Hosteller: Represents students living in hostels with hostel ID and name, including applying for outpass and paying hostel fees.
  - Fees: Represents fee payment information and related payment operations.
  - CRC_ATPC: Manages placement and internship applications, centralized placements, and resume uploads.
  - OD (On-Duty): Handles applications and approvals or rejections of on-duty requests by students.
  - HOD (Head of Department): Responsible for approvals (like APW) and creating timetables for their department.
  - PL (Program Leader): Oversees programs, approves APW, views student details, and manages timetables.
  - Course: Contains course information such as code and name, and updates syllabus.
  - Examination: Manages exam schedules and publishes results, tracking percentages and internal marks.

  Components:

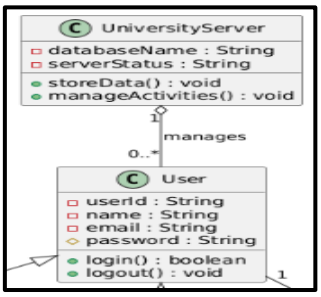
  - Association: Association shows direct interaction between classes, such as UniversityServer managing users, User being authenticated by AuthServer, faculty teaching courses, and students interacting with fees, CRC_ATPC, and OD.



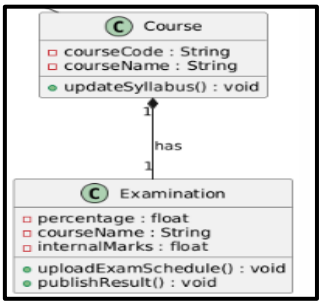
- Multiplicity: Multiplicity defines the number of objects involved in a relationship, for example one server managing many users, one faculty teaching multiple courses, and one course having one examination.



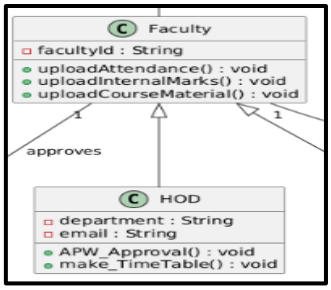
- Aggregation: Aggregation represents a weak "has-a" relationship where the UniversityServer manages multiple Users that can exist independently.



- Composition: Represents a strong "has-a" relationship where the child cannot exist without the parent, such as an Examination being part of a Course.



- Generalization: Generalization shows inheritance relationships, where Student and Faculty inherit from User, and HOD and PL inherit from Faculty.



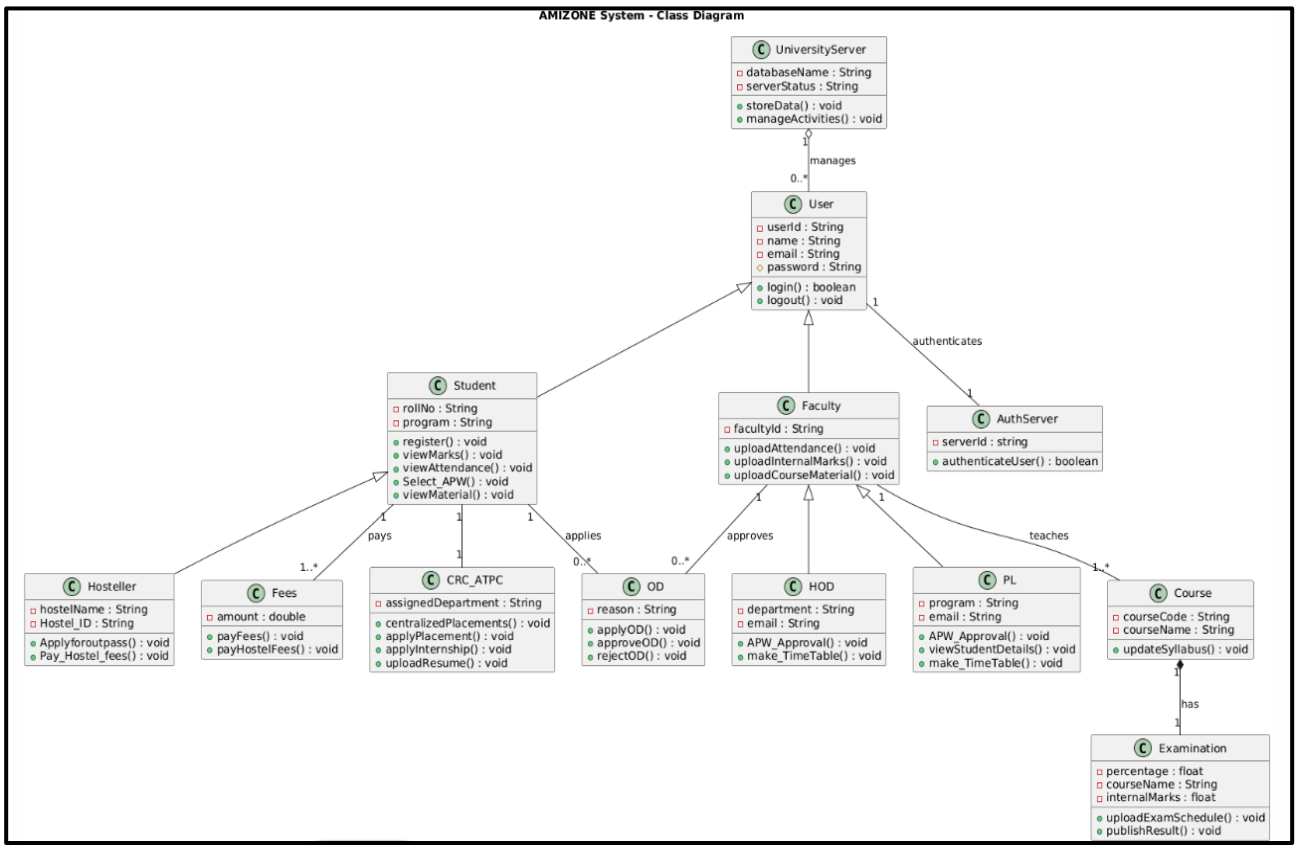
- **<u>CODE:</u>**

```
@startuml
title AMIZONE System - Class Diagram
' Base User Class
class User {
  - userId : String
```

```
  - name : String
  - email : String
  # password : String
  + login() : boolean
  + logout() : void
}
' User Hierarchy
class Student {
  - rollNo : String
  - program : String
  + register() : void
  + viewMarks() : void
  + viewAttendance() : void
  + Select_APW() : void
  + viewMaterial() : void
}
class Hosteller{
  - hostelName : String
  - Hostel_ID : String
  + Applyforoutpass() : void
  + Pay_Hostel_fees() : void
 }
class Faculty {
  - facultyId : String
  + uploadAttendance() : void
  + uploadInternalMarks() : void
  + uploadCourseMaterial() : void
}
class HOD{
    - department : String
    - email : String
    + APW_Approval() : void
    + make_TimeTable() : void
}
class PL{
    - program : String
    - email : String
    + APW_Approval() : void
    + viewStudentDetails() : void
    + make_TimeTable() : void
}
User <|-- Student
Student <|-- Hosteller
User <|-- Faculty
```

```
Faculty <|-- HOD
Faculty <|-- PL
' Academic Modules
class Course {
  - courseCode : String
  - courseName : String
  + updateSyllabus() : void
}
class Examination{
  - percentage : float
  - courseName : String
  - internalMarks : float
  + uploadExamSchedule() : void
  + publishResult() : void
}
' Composition: exam cannot exist without course
Course "1" *-- "1" Examination : has
' Association with multiplicity
Faculty "1" -- "1..*" Course : teaches
'OD
class OD {
  - reason : String
  + applyOD() : void
  + approveOD() : void
  + rejectOD() : void
}
' Association with multiplicity
Student "1" -- "0..*" OD : applies
Faculty "1" -- "0..*" OD : approves
' Fees
class Fees {
  - amount : double
  + payFees() : void
  + payHostelFees() : void
}
' Associations with multiplicity
Student "1" -- "1..*" Fees : pays
' Placement & Internship
class CRC_ATPC {
  - assignedDepartment : String
  + centralizedPlacements() : void
  + applyPlacement() : void
  + applyInternship() : void
  + uploadResume() : void
```

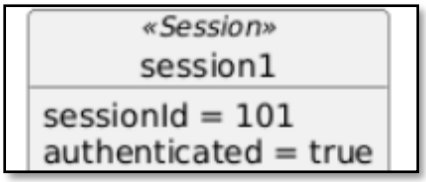
```
}
' Authentication & Server
class AuthServer {
  - serverId : string
  + authenticateUser() : boolean
}
class UniversityServer {
  - databaseName : String
  - serverStatus : String
  + storeData() : void
  + manageActivities() : void
}
' Associations with multiplicity
User "1" -- "1" AuthServer : authenticates
' Aggregation: UniversityServer has Users, but Users can exist independently
UniversityServer "1" o-- "0..*" User : manages
Student "1" -- "1" CRC_ATPC
@enduml
```



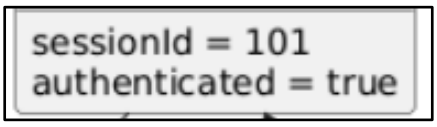
LAB Assignment 3

- **Theory:** An Object Diagram is a type of structural diagram in UML that represents the dynamic view of a system at a particular moment in time. It shows instances of classes (objects), their attribute values, and the links between objects. Object diagrams are used to illustrate real examples of class diagrams and help in understanding how objects interact during system execution.
  Object diagrams help in visualizing system behavior, validating class diagrams, and understanding object interactions at runtime.

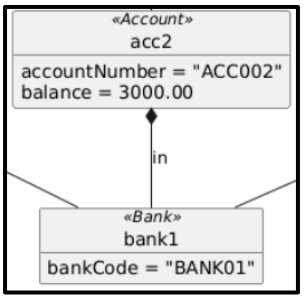
  Components of Activity Diagram:
  - Object: Represented by a rectangle with the object name and class name in the first part, and attribute values in second part.



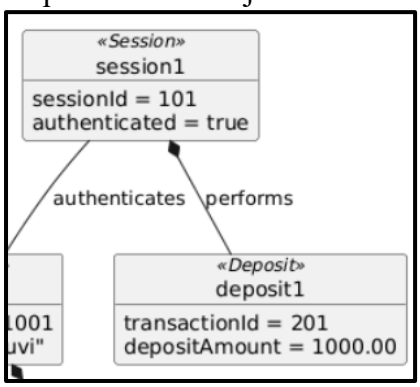
  - Attributes: Shown inside the object with assigned values, representing the current state of the object.



  - Links: Represented by solid lines connecting objects, showing relationships or interactions between object instances.



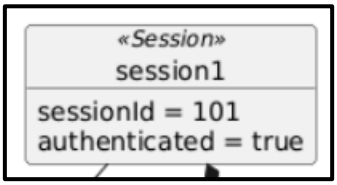
  - Aggregation: Represents a weak "has-a" relationship where objects can exist independently. In the ATM system, the Bank aggregates multiple accounts, meaning accounts can exist without a specific Bank object instance.



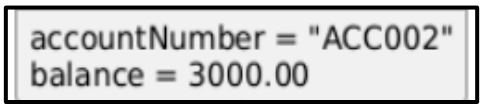
3(a)

- **Aim:** Object Diagram of ATM System.
  The ATM Object Diagram represents the ATM system at a particular moment by showing its objects, their attribute values, and the links among them. It explains how different instances of classes interact with each other to support banking operations such as authentication and transactions.
  Objects:
  - Session Objects (session1, session2): Represent individual ATM sessions managing user authentication and controlling the execution of transactions.
  - USER Objects (user1, user2): Represent customers with specific details like user ID and name, linked to their accounts.
  - Account Objects (acc1, acc2): Represent bank accounts with actual balances and account numbers, associated with users and the bank.
  - Transaction Objects (deposit1, withdraw1, balanceInquiry1): Represent specific transactions performed within sessions, each having transaction IDs and related details.
  - Bank Object (bank1): Represents the banking institution managing multiple accounts and providing banking services.

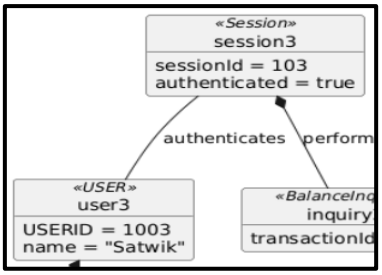
  Components:
  - Object: Represented by a rectangle with the object name and class name in the first part (<<*Session*>> : session1), and attribute values in second part.



  - Attributes: Shown inside the object with assigned values, representing the current state of the object, for example userID, accountNumber, balance, or transactionAmount.



  - Links: Represented by solid lines connecting objects, showing relationships or interactions between object instances such as authentication, ownership of accounts, and transaction execution.

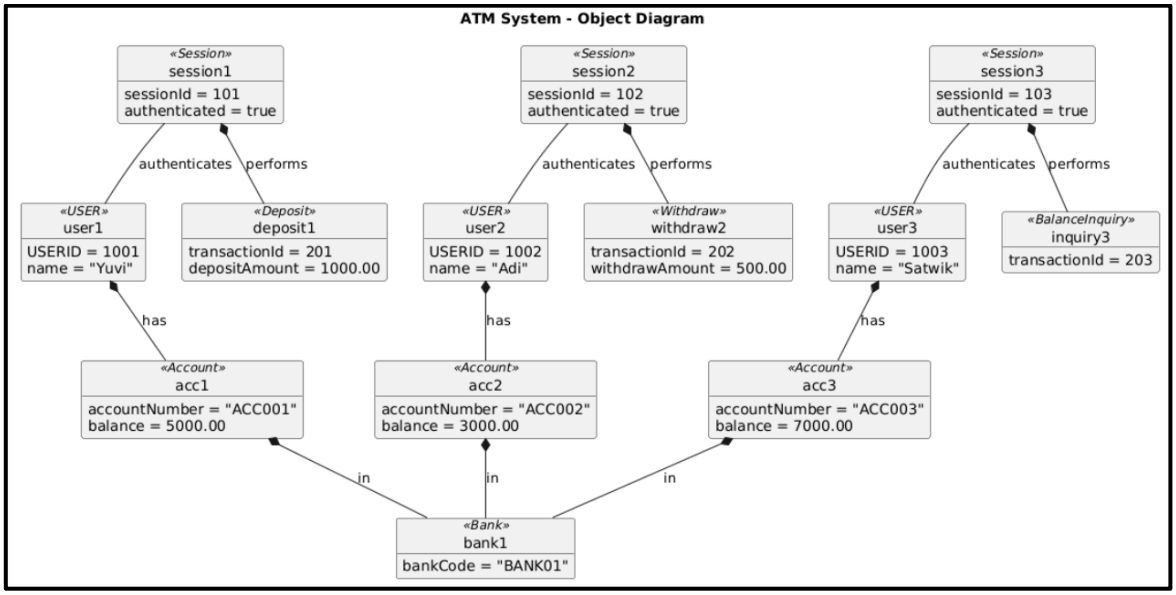


- **CODE:**

```
@startuml
title ATM System – Object Diagram
object session1 <<Session>> {
  sessionId = 101
```

```
  authenticated = true
}
object session2 <<Session>> {
  sessionId = 102
  authenticated = true
}
object session3 <<Session>> {
  sessionId = 103
  authenticated = true
}
object user1 <<USER>> {
  USERID = 1001
  name = "Yuvi"
}
object user2 <<USER>> {
  USERID = 1002
  name = "Adi"
}
object user3 <<USER>> {
  USERID = 1003
  name = "Satwik"
}
session1 -- user1 : authenticates
session2 -- user2 : authenticates
session3 -- user3 : authenticates
object deposit1 <<Deposit>> {
  transactionId = 201
  depositAmount = 1000.00
}
object withdraw2 <<Withdraw>> {
  transactionId = 202
  withdrawAmount = 500.00
}
object inquiry3 <<BalanceInquiry>> {
  transactionId = 203
}
session1 *-- deposit1 : performs
session2 *-- withdraw2 : performs
session3 *-- inquiry3 : performs
object bank1 <<Bank>> {
  bankCode = "BANK01"
}
object acc1 <<Account>> {
  accountNumber = "ACC001"
  balance = 5000.00
}
```

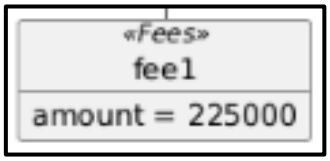
```
object acc2 <<Account>> {
  accountNumber = "ACC002"
  balance = 3000.00
}
object acc3 <<Account>> {
  accountNumber = "ACC003"
  balance = 7000.00
}
acc1 *-- bank1 : in
acc2 *-- bank1 : in
acc3 *-- bank1 : in
user1 *-- acc1 : has
user2 *-- acc2 : has
user3 *-- acc3 : has
@enduml
```



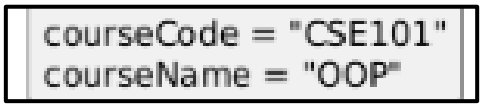
3(b)

- **Aim:** Object Diagram of Amizone System.
  The AMIZONE Object Diagram represents the AMIZONE system at a particular moment by showing its objects, their attribute values, and the links among them. It explains how different instances of classes interact with each other to support academic operations such as enrollment, fee payment, course management, and approvals.
  Objects:
  - Faculty Objects (fac1, fac2): Represent faculty members with details like user ID, name, email, and department.
  - Student Objects (stu1, stu2, stu3): Represent students with specific details like user ID, name, roll number, program, hostel ID/name, and section.
  - HOD Object (fac2): Represents the Head of Department with user ID, name, department, and email.
  - PL Object (fac3): Represents the Program Leader with user ID, name, program, and email.
  - Course Objects (course1, course2, course3): Represent courses with course codes and course names.
  - Examination Objects (exam1, exam2, exam3): Represent examinations with course names, internal marks, and percentage.
  - Fees Objects (fee1, fee2, fee3): Represent fee payments with amount and hostel fees where applicable.
  - CRC_ATPC Object (crc1): Represents the department placement/approval committee with assigned department.
  - OD Object (od1): Represents on-duty approval with a reason.

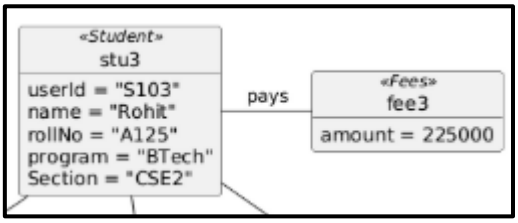
  Components:
  - Object: Represented by a rectangle with the object name and class name in the first part (<<Student>> stu1), and attribute values in the second part.



  - Attributes: Shown inside the object with assigned values, representing the current state of the object, for example userId, courseCode, amount, internalMarks, or reason.



  - Links: Represented by solid lines connecting objects, showing relationships or interactions between object instances such as teaching, enrollment, fee payment, application, approval, and course-examination linkage.

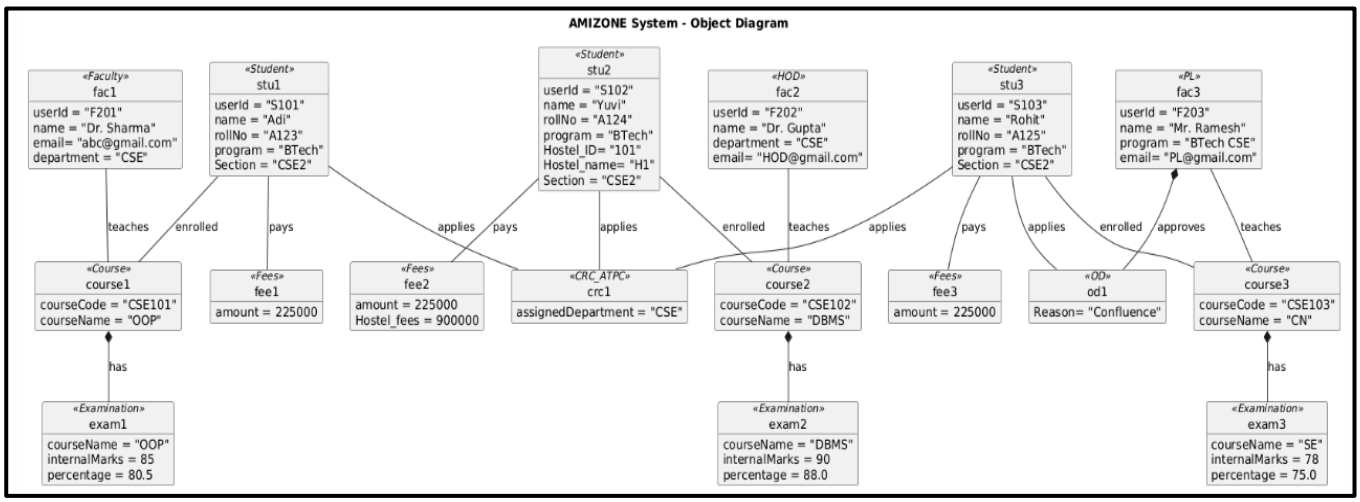


- **CODE:**

```
@startuml
title AMIZONE System - Object Diagram
object fac1 <<Faculty>> {
   userId = "F201"
   name = "Dr. Sharma"
   email= "abc@gmail.com"
   department = "CSE"
}
object fac2 <<HOD>> {
   userId = "F202"
   name = "Dr. Gupta"
   department = "CSE"
   email= "HOD@gmail.com"
}
object fac3 <<PL>> {
   userId = "F203"
   name = "Mr. Ramesh"
   program = "BTech CSE"
   email= "PL@gmail.com"
}
object stu1 <<Student>> {
   userId = "S101"
   name = "Adi"
   rollNo = "A123"
   program = "BTech"
   Section = "CSE2"
}
object stu2 <<Student>> {
   userId = "S102"
   name = "Yuvi"
   rollNo = "A124"
   program = "BTech"
   Hostel_ID= "101"
   Hostel_name= "H1"
   Section = "CSE2"
}
object stu3 <<Student>> {
   userId = "S103"
   name = "Rohit"
   rollNo = "A125"
   program = "BTech"
   Section = "CSE2"
}
object course1 <<Course>> {
   courseCode = "CSE101"
   courseName = "OOP"
```

```
}
object course2 <<Course>> {
   courseCode = "CSE102"
   courseName = "DBMS"
}
object course3 <<Course>> {
   courseCode = "CSE103"
   courseName = "CN"
}
fac1 -- course1 : teaches
fac2 -- course2 : teaches
fac3 -- course3 : teaches
stu1 -- course1 : enrolled
stu2 -- course2 : enrolled
stu3 -- course3 : enrolled
object exam1 <<Examination>> {
   courseName = "OOP"
   internalMarks = 85
   percentage = 80.5
}
object exam2 <<Examination>> {
   courseName = "DBMS"
   internalMarks = 90
   percentage = 88.0
}
object exam3 <<Examination>> {
   courseName = "SE"
   internalMarks = 78
   percentage = 75.0
}
object od1<<OD>> {
   Reason= "Confluence"
}
stu3 -- od1 : applies
fac3  *-- od1 : approves
course1 *-- exam1 : has
course2 *-- exam2 : has
course3 *-- exam3 : has
object fee1 <<Fees>> {
   amount = 225000
}
object fee2 <<Fees>> {
   amount = 225000
   Hostel_fees = 900000
}
object fee3 <<Fees>> {
```

```
    amount = 225000
}
stu1 -- fee1 : pays
stu2 -- fee2 : pays
stu3 -- fee3 : pays
object crc1 <<CRC_ATPC>> {
    assignedDepartment = "CSE"
}
stu1 -- crc1 : applies
stu2 -- crc1 : applies
stu3 -- crc1 : applies
@enduml
```

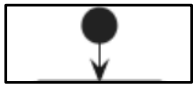


LAB Assignment 4

- **Theory:** An Activity Diagram is a type of behavioral diagram in UML that represents the workflow or flow of activities within a system. It shows the sequence of actions, decisions, and parallel processes involved in carrying out a specific functionality. Activity diagrams are used to model dynamic aspects of a system and illustrate how control flows from one activity to another during execution.
  Activity diagrams help in understanding system behavior, modeling business processes, identifying decision points, and visualizing the flow of control in complex operations.

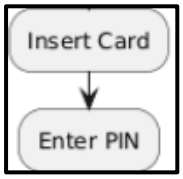
  Components of Activity Diagram:
  - Activity: Represents a specific action or operation performed in the system. It is shown as a rounded rectangle and describes a step in the workflow.

  - Initial Node: Indicates the starting point of the activity diagram. It is represented by a filled black circle.



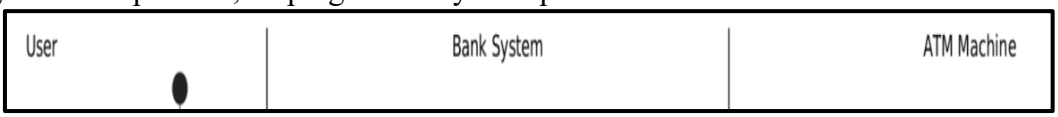
  - Final Node: Represents the termination of the activity flow. It is shown as a filled black circle surrounded by a hollow circle.

  - Control Flow: Shown as arrows connecting activities, indicating the order in which actions are executed.



  - Decision Node: Represents a point where a choice is made based on conditions. It is shown as a diamond shape with guard conditions on outgoing flows.

  - Swimlanes: Used to divide activities based on responsibilities of different actors or system components, helping to clarify who performs each action.



4(a)

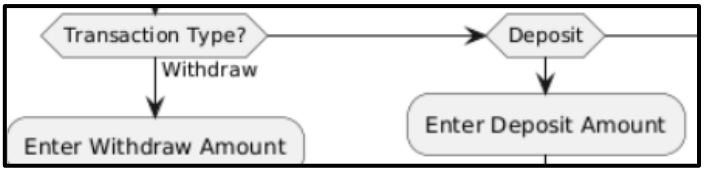
- **Aim:** Activity Diagram of ATM System.
  The Activity Diagram represents the dynamic workflow of the ATM system, showing the sequence of activities, decisions, and flows that occur during user authentication and banking transactions like withdrawals, deposits, and transfers. It helps visualize how different components interact over time to complete operations, making system behavior clear and supporting analysis, validation, and design.

  Activities:
  - Insert Card: User inserts their ATM card to start the session.
  - Enter PIN: User enters their PIN for authentication.
  - Display Transaction Menu: ATM shows available transaction options.
  - Select Transaction: User selects the type of transaction (Withdraw, Deposit, Transfer).
  - Enter Withdraw Amount: User inputs the amount to withdraw.
  - Check Balance: Bank system verifies if sufficient funds are available.
  - Debit Account: Bank system deducts the withdrawn amount from the account.
  - Dispense Cash: ATM dispenses cash for a withdrawal transaction.
  - Show Insufficient Balance: ATM notifies the user if funds are insufficient.
  - Enter Deposit Amount: User inputs the amount to deposit.
  - Credit Account: Bank system adds the deposited amount to the account.
  - Enter Target Account: User inputs the account number for transfer.
  - Enter Transfer Amount: User inputs the amount to transfer.
  - Debit Source Account: Bank system deducts transfer amount from source account.
  - Credit Target Account: Bank system adds transfer amount to target account.
  - Update Bank Database: Bank system updates account records after each transaction.
  - Show Transaction Result: ATM shows confirmation of completed transaction.
  - End Session: User ends the session.
  - Eject Card: ATM returns the card at the end of session or after invalid PIN entry.
  - Show Invalid PIN: ATM notifies user if PIN is incorrect..

  Components:
  - Activities: Actions performed by the system or users, such as Insert Card, Enter PIN, Select Transaction, Process Withdraw/Deposit, Update Account, and Eject Card.

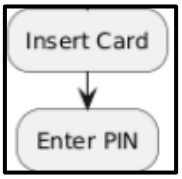
  - Swimlanes: Divide responsibilities among the User (inputs), ATM Machine (interface), and Bank System (processing).

  - Decision Nodes (Diamonds): Represent branching points for choices, like PIN Valid?, Transaction Type, or Sufficient Balance?.



- Start/End Nodes: Indicate the beginning and termination of the workflow.

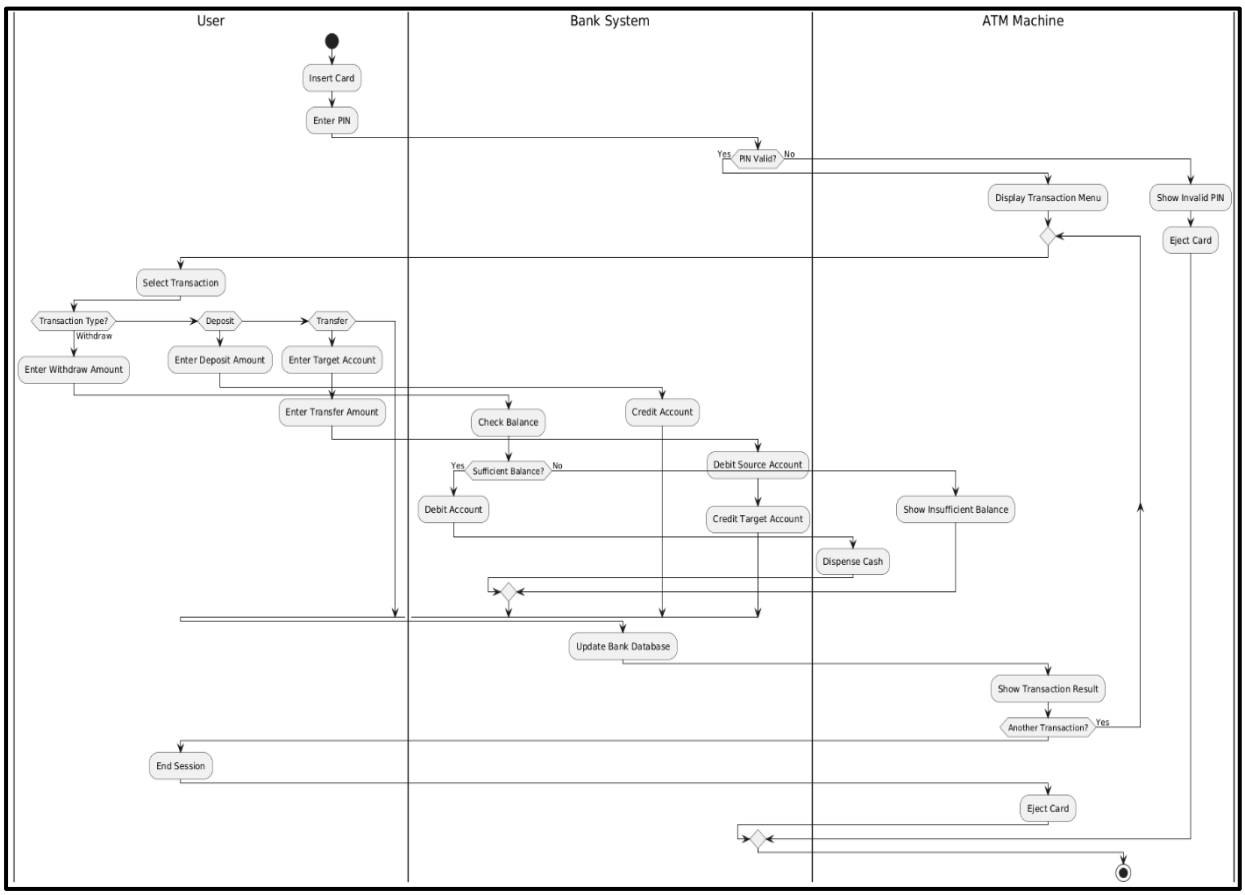
- Transitions: Arrows showing the flow from one activity to another.



- **<u>CODE:</u>**

```
@startuml
|User|
start
:Insert Card;
:Enter PIN;
|Bank System|
if (PIN Valid?) then (Yes)
  |ATM Machine|
  :Display Transaction Menu;
  repeat
    |User|
    :Select Transaction;
    if (Transaction Type?) then (Withdraw)
      :Enter Withdraw Amount;
      |Bank System|
      :Check Balance;
      if (Sufficient Balance?) then (Yes)
        :Debit Account;
        |ATM Machine|
        :Dispense Cash;
      else (No)
        |ATM Machine|
        :Show Insufficient Balance;
      endif
    elseif (Deposit)
      |User|
      :Enter Deposit Amount;
      |Bank System|
      :Credit Account;
    elseif (Transfer)
      |User|
      :Enter Target Account;
      :Enter Transfer Amount;
      |Bank System|
```

```
    :Debit Source Account;
    :Credit Target Account;
  endif
  |Bank System|
  :Update Bank Database;
  |ATM Machine|
  :Show Transaction Result;
 repeat while (Another Transaction?) is (Yes)
 |User|
 :End Session;
 |ATM Machine|
 :Eject Card;
else (No)
 |ATM Machine|
 :Show Invalid PIN;
 :Eject Card;
endif
stop
@enduml
```



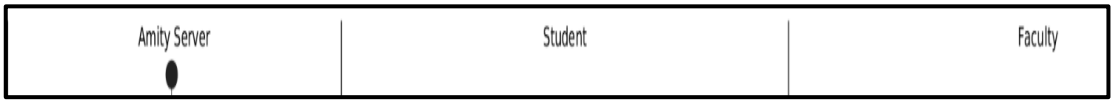
4(b)

- **Aim:** Activity Diagram of Amizone System.
  The AMIZONE Activity Diagram represents the AMIZONE system's dynamic workflow by showing the sequence of activities, decisions, and flows involved in academic operations such as authentication, enrollment, fee payment, course management, and approvals. It explains how different actors interact with the system to perform specific tasks and how control flows between activities.

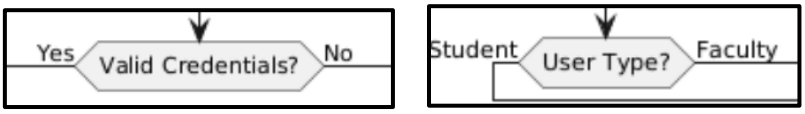
  Activities:
  - Open Portal: User opens the Amizone portal to start a session.
  - Enter Credentials: Users (students or faculty) enter their login details.
  - Login Successful / Authentication Error: The system validates credentials.
  - Select Service: Students or faculty choose the operation they want to perform.
  - View Attendance / View Marks: Students access their attendance or marks.
  - Apply OD / Register APW / Pay Fees: Students perform requests or payments.
  - Upload Attendance / Upload Marks / Approve OD/APW / Upload Material: Faculty perform academic or approval-related tasks.
  - Logout: The system ends the session after tasks are completed.

  Components:
  - Activities: Represented as a rectangle with rounded edges showing an action being performed, for example Enter Credentials, View Attendance, or Upload Marks.

  - Swimlanes: Vertical partitions dividing responsibilities among Student, Faculty, and Amity Server, showing which actor performs each activity.

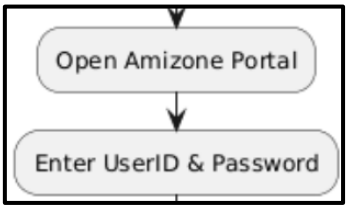


  - Decision Nodes (Diamonds): Represent branching points where flow diverges based on conditions such as Valid Credentials?, User Type?, Service Selected?, or OD/APW Approved?.



  - Start/End Nodes: Represent the beginning and termination of the workflow.

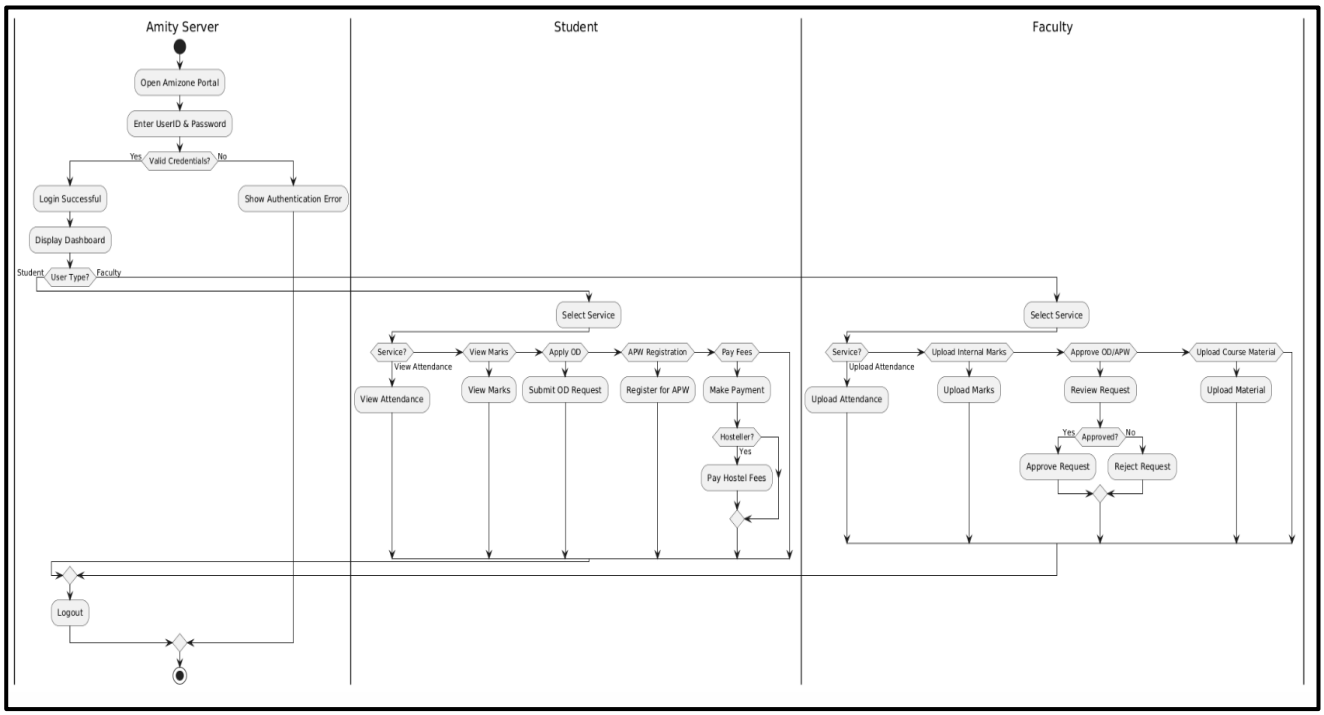
  - Transitions: Show the flow of control from one activity to another.



- **CODE:**

```
@startuml
|Amity Server|
start
:Open Amizone Portal;
:Enter UserID & Password;

if (Valid Credentials?) then (Yes)

  :Login Successful;
  :Display Dashboard;

  if (User Type?) then (Student)

      |Student|
      :Select Service;

      if (Service?) then (View Attendance)
        :View Attendance;

      elseif (View Marks)
        :View Marks;

      elseif (Apply OD)
        :Submit OD Request;

      elseif (APW Registration)
        :Register for APW;

      elseif (Pay Fees)
        :Make Payment;
        if (Hosteller?) then (Yes)
          :Pay Hostel Fees;
        endif
      endif

  else (Faculty)

      |Faculty|
      :Select Service;

      if (Service?) then (Upload Attendance)
        :Upload Attendance;

      elseif (Upload Internal Marks)
        :Upload Marks;
```

```
    elseif (Approve OD/APW)
      :Review Request;
      if (Approved?) then (Yes)
        :Approve Request;
      else (No)
        :Reject Request;
      endif

    elseif (Upload Course Material)
      :Upload Material;
    endif
  endif

  |Amity Server|
  :Logout;

else (No)
  |Amity Server|
  :Show Authentication Error;
endif

stop
@enduml
```



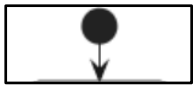
LAB Assignment 5

- **<u>Theory:</u>** A State Diagram (also known as a State Machine Diagram) is a type of behavioral diagram in UML (Unified Modeling Language) that represents the different states of an object and the transitions between those states during its lifecycle. It shows how an object changes its state in response to events and conditions. State diagrams are used to model the dynamic behavior of a system by illustrating how an object reacts to internal or external events. They help in understanding object life cycles, modeling reactive systems, and visualizing how states change over time based on triggers and actions.

  <u>Components of State Diagram:</u>

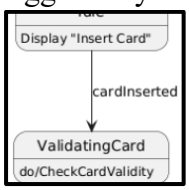
  - State: Represents a condition or situation during the life of an object when it satisfies certain conditions, performs an activity, or waits for an event. It is shown as a rounded rectangle with the state name inside.

  - Initial State: Indicates the starting point of the state machine. It is represented by a filled black circle.

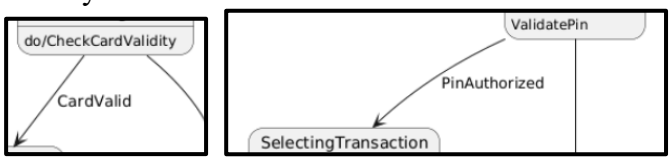


  - Final State: Represents the end of the state machine's execution. It is shown as a filled black circle surrounded by a hollow circle.

  - Transition: Shows the movement from one state to another. It is represented by an arrow connecting two states and is triggered by an event.

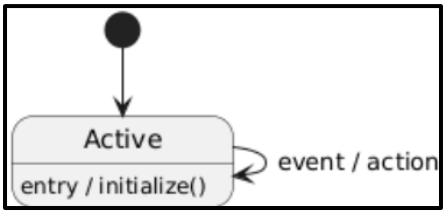


  - Event: An occurrence that triggers a transition from one state to another. Events can be external (user actions) or internal (system-generated).
  - Guard Condition: A Boolean condition written in square brackets on a transition. The transition occurs only if the condition evaluates to true.

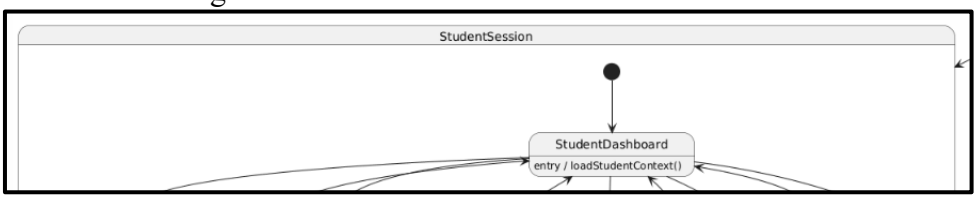


  - Action: An activity that is executed as a result of a transition. It is usually written along the transition arrow after the event.

  - Self-Transition: A transition where a state loops back to itself, indicating that the object remains in the same state after an event.



- Composite State: A state that contains one or more substates, used to represent complex behavior within a single state.



5(a)

- **Aim:** State Diagram of ATM System.

  The State Diagram shows the dynamic behavior of the ATM system by representing the different states it goes through during an ATM session. It illustrates how the system responds to user actions like inserting a card, entering a PIN, selecting a transaction, and completing operations such as withdrawal, deposit, or balance inquiry.

  It helps in understanding the ATM session lifecycle by showing how transitions occur based on events and conditions. The diagram ensures proper handling of authentication, transaction processing, and session termination, making system behavior clear and easy to analyze.

  States:

  • Initial State ([*]): Represents the starting point of the ATM system before any user interaction begins.

  • Idle State: The ATM waits for user interaction.

  Action: Displays "Insert Card" message.

  • Validating Card State : Triggered when a card is inserted.

  Action: Checks whether the card is valid (do/CheckCardValidity).

  If the card is invalid → Card is ejected and system ends.

  If valid → Moves to authentication state.

  • Authenticating State: The ATM verifies the user's PIN (ValidatePin).

  If PIN is correct → Proceeds to transaction selection.

  If PIN is incorrect → Card is ejected and system ends.

  • Selecting Transaction State: Displays transaction menu options.

  User can select: Withdrawal, Deposit, Balance Inquiry

  • Withdrawing State :User enters amount → System checks balance → Dispenses cash.

  After completion, moves to Exit state.

  • Depositing State: User enters deposit amount → System credits account.

  After completion, moves to Exit state.

  • Inquiry State: System retrieves and displays account balance.

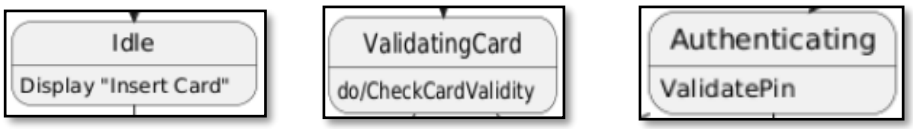
  After completion, moves to Exit state.

  • Exit State: Card is ejected and session is terminated.

  • Final State ([*]):Represents the end of the ATM session.

  Components:

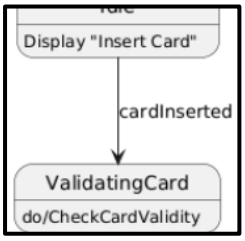
  - State: Represents a condition of the ATM during operation (Idle, ValidatingCard, Authenticating, Withdrawing, Depositing, Inquiry, Exit).



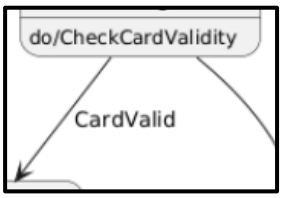
  - Initial State: Shown as a filled black circle. Indicates where the ATM session begins.

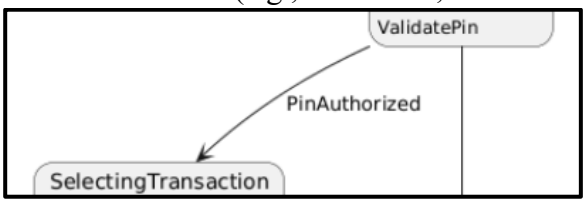
  - Final State: Shown as a filled black circle with a surrounding hollow circle. Indicates the end of the ATM session.

- Transition: Arrows that show movement between states (e.g., cardInserted, CardValid, PinAuthorized.



- Event: Triggers that cause transitions (e.g., inserting card, cardInserted, selecting transaction).
- Guard Condition: Conditions that control transitions (e.g., CardValid, PinAuthorized).



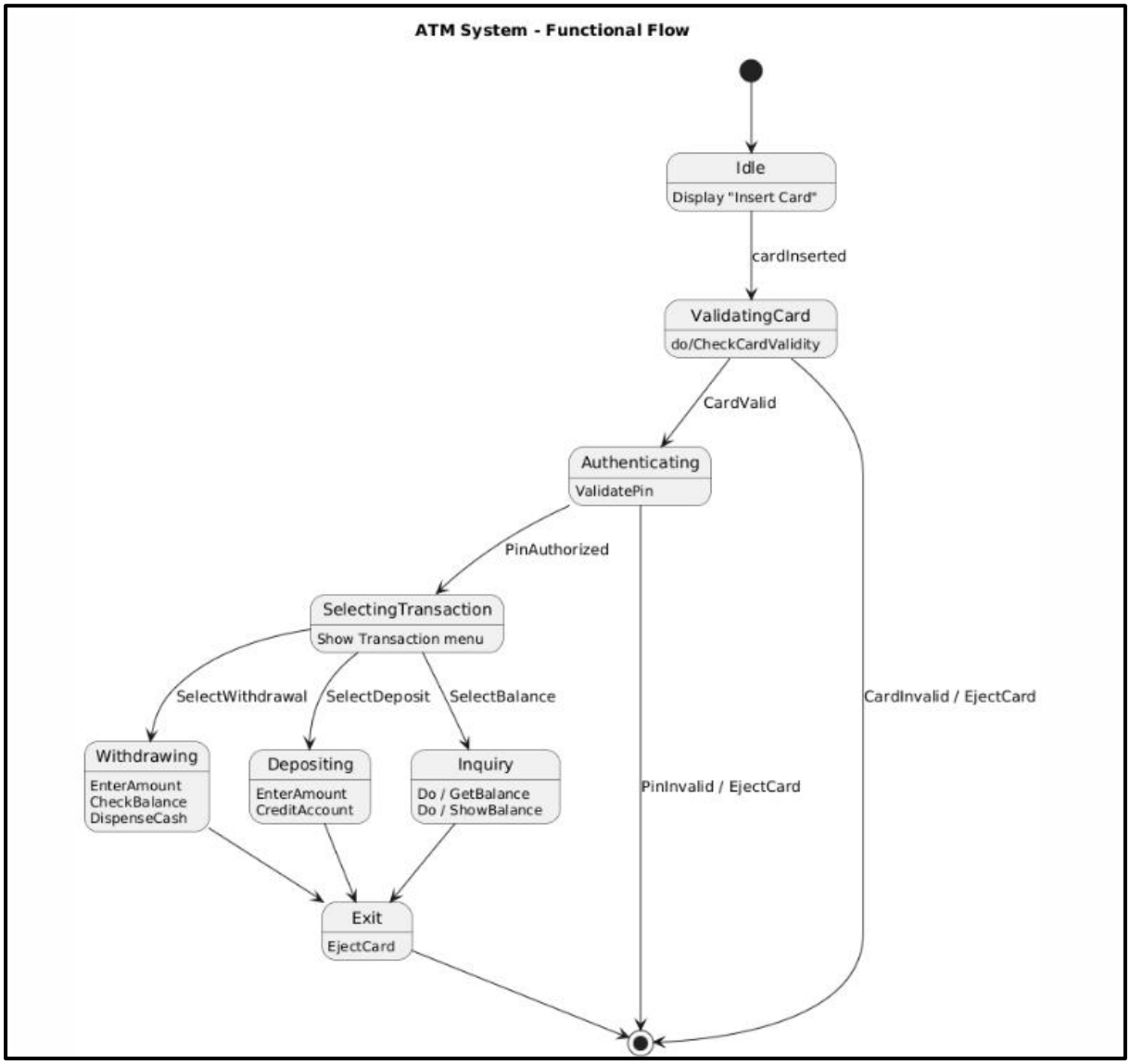


- Action: Operations performed within a state (e.g., CheckCardValidity, ValidatePin, DispenseCash).

- **<u>CODE:</u>**

```
@startuml
title ATM System - Functional Flow

[*] --> Idle

state Idle : Display "Insert Card"
Idle --> ValidatingCard : cardInserted

state ValidatingCard : do/CheckCardValidity
ValidatingCard --> Authenticating : CardValid
ValidatingCard --> [*] : CardInvalid / EjectCard

state Authenticating : ValidatePin
Authenticating --> SelectingTransaction : PinAuthorized
Authenticating --> [*] : PinInvalid / EjectCard

state SelectingTransaction : Show Transaction menu
SelectingTransaction --> Withdrawing : SelectWithdrawal
SelectingTransaction --> Depositing : SelectDeposit
SelectingTransaction --> Inquiry : SelectBalance

state Withdrawing : EnterAmount
Withdrawing : CheckBalance
Withdrawing : DispenseCash
Withdrawing --> Exit
```

```
state Depositing : EnterAmount
Depositing : CreditAccount
Depositing --> Exit

state Inquiry : Do / GetBalance
Inquiry : Do / ShowBalance
Inquiry --> Exit

state Exit : EjectCard
Exit --> [*]
@enduml
```



5(b)

- **Aim:** State Diagram of Amizone System.

The State Diagram represents the dynamic behavior of the AMIZONE system by showing the different states it goes through during a user session. It illustrates how the system processes login credentials, authenticates the user, and directs them to either the Student or Faculty session based on their role. It helps in understanding the flow from login to authentication, module navigation, and logout. The diagram clearly shows role-based transitions, handling of authentication success or failure, and session termination, making system behavior easy to understand and analyze..

States:

• Initial State ([*]): Represents the starting point of the system before login

• Idle State: Displays login page.

Entry Action: display "Enter Login Details".

• Validating Card State : Triggered when a card is inserted.

Action: Checks whether the card is valid (do/CheckCardValidity).

If the card is invalid → Card is ejected and system ends.

If valid → Moves to authentication state.

• Authenticating State: Validates user credentials (authenticateUser).

- If authentication fails → Returns to Idle and shows error.
- If authentication succeeds → Moves to StudentSession or FacultySession.

StudentSession (Composite State): Represents the complete session of a student user.

• StudentDashboard : Acts as the central navigation point for student modules.

• AcademicModule : Viewing attendance and marks, Returns to dashboard after completion.

• HostelModule: Apply for outpass, Pay hostel fees.

• FeeModule: Handles fee transaction processing.

• PlacementModule: Upload resume, Apply for internship (CRC portal).

• Logout: StudentDashboard → Exit state.

FacultySession (Composite State): Represents the complete session of a faculty member.

• FacultyDashboard: Acts as the central navigation point for Faculty.

• ManagementModule: Upload attendance, Upload internal marks, AdministrativeTasks (for HOD/PL role).

• Logout: FacultyDashboard → Exit state.

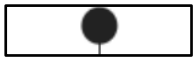
• Exit State: Represents the termination of the session.

• Final State ([*]): Indicates the end of the system execution for that session.

Components:

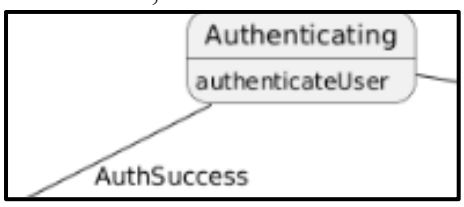
- State: Represents distinct conditions during the system's operation, such as Idle, Authenticating, StudentDashboard, FacultyDashboard, and various modules.

- Initial State: Shown as a filled black circle. Indicates starting point before user login.

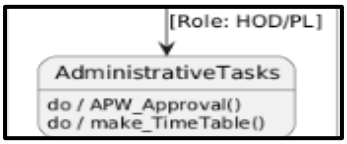


- Final State: Shown as a filled black circle with a surrounding hollow circle. end of a user session after logout.

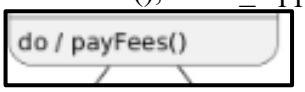
- Transition: Arrows showing movement from one state to another triggered by events, e.g., login(credentials), AuthSuccess, authFailed / show error, logout().



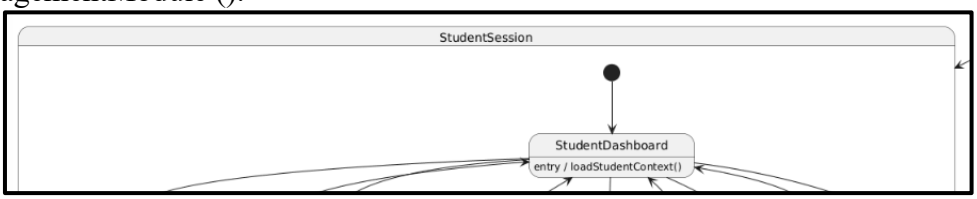
- Event: User or system triggers that cause state transitions, such as login attempt, successful authentication, logout command, or module navigation commands.
- Guard Condition: Conditions controlling transitions, e.g., [if Hosteller] and [Role: HOD/PL].



- Action: Operations executed inside states, such as authenticateUser, viewAttendance(), payFees(), uploadAttendance(), APW_Approval().



- Composite State: States containing nested substates to represent complex workflows, like StudentSession containing multiple modules, and FacultySession containing ManagementModule ().

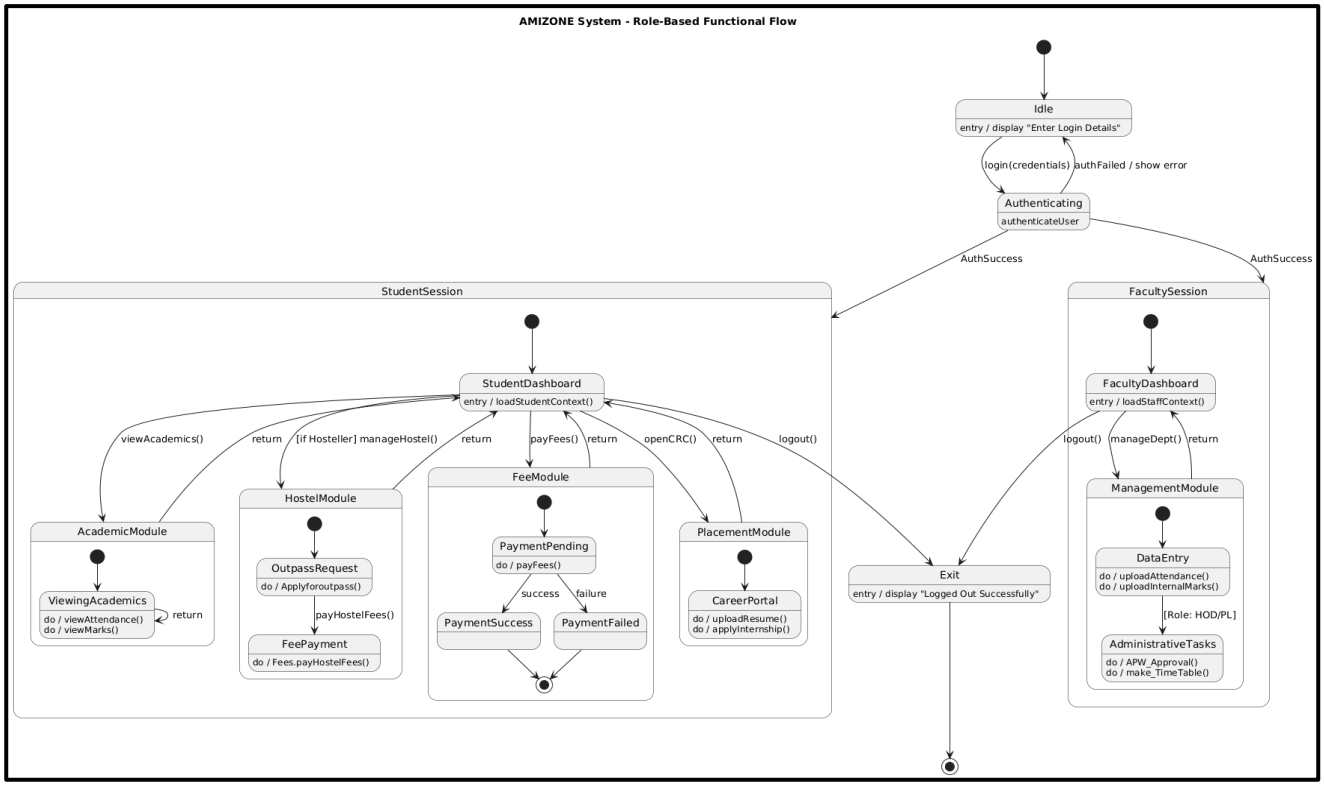


- **CODE:**

```
@startuml
title AMIZONE System - Role-Based Functional Flow
[*] --> Idle
state Idle : entry / display "Enter Login Details"
Idle --> Authenticating : login(credentials)
state Authenticating : authenticateUser
Authenticating --> StudentSession : AuthSuccess
Authenticating --> FacultySession : AuthSuccess
Authenticating --> Idle : authFailed / show error
state StudentSession {
   [*] --> StudentDashboard
   StudentDashboard : entry / loadStudentContext()
   state AcademicModule {
      [*] --> ViewingAcademics
      ViewingAcademics : do / viewAttendance()
```

```
        ViewingAcademics : do / viewMarks()
        ViewingAcademics --> ViewingAcademics : return
    }
    state HostelModule {
        [*] --> OutpassRequest
        OutpassRequest : do / Applyforoutpass()
        OutpassRequest --> FeePayment : payHostelFees()
        FeePayment : do / Fees.payHostelFees()
    }
    state FeeModule {
        [*] --> PaymentPending
        PaymentPending : do / payFees()
        PaymentPending --> PaymentSuccess : success
        PaymentPending --> PaymentFailed : failure
        PaymentSuccess --> [*]
        PaymentFailed --> [*]
    }
   state PlacementModule {
        [*] --> CareerPortal
        CareerPortal : do / uploadResume()
        CareerPortal : do / applyInternship()
    }
    StudentDashboard --> AcademicModule : viewAcademics()
    StudentDashboard --> HostelModule : [if Hosteller] manageHostel()
    StudentDashboard --> FeeModule : payFees()
    StudentDashboard --> PlacementModule : openCRC()
    AcademicModule --> StudentDashboard : return
    HostelModule --> StudentDashboard : return
    FeeModule --> StudentDashboard : return
    PlacementModule --> StudentDashboard : return
    StudentDashboard --> Exit : logout()
}
state FacultySession {
    [*] --> FacultyDashboard
    FacultyDashboard : entry / loadStaffContext()
    state ManagementModule {
        [*] --> DataEntry
        DataEntry : do / uploadAttendance()
        DataEntry : do / uploadInternalMarks()
        state AdministrativeTasks {
            AdministrativeTasks : do / APW_Approval()
            AdministrativeTasks : do / make_TimeTable()
        }
        DataEntry --> AdministrativeTasks : [Role: HOD/PL]
    }
    FacultyDashboard --> ManagementModule : manageDept()
```

```
    ManagementModule --> FacultyDashboard : return
    FacultyDashboard --> Exit : logout()
}
state Exit : entry / display "Logged Out Successfully"
Exit --> [*]
@enduml
```

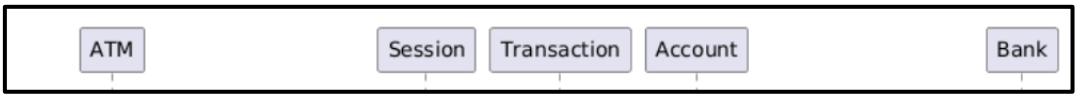


LAB Assignment 6

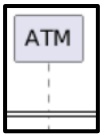
- **Theory:** A Sequence Diagram is a type of behavioral diagram in UML (Unified Modeling Language) that shows how objects interact with each other in a particular order to complete a task. It represents the sequence of messages exchanged between objects over time.
  In a sequence diagram, objects are arranged horizontally, and time flows from top to bottom. It helps in understanding how different parts of a system communicate to perform a function and is commonly used to model use cases, system behavior, and workflows.

  Components of Sequence Diagram:
  - Actor: Represents a user or external system that interacts with the system. It is shown as a stick figure.

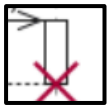
  - Object (Participant): Represents an instance that takes part in the interaction. It is shown as a rectangle with the object name.



  - Lifeline: A vertical dashed line below an object that shows the time the object exists in the interaction.



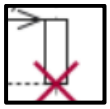
  - Activation Bar: A thin rectangle on the lifeline showing when an object is performing an action.



  - Message: Communication between objects represented by arrows. It shows how one object sends a request or information to another.

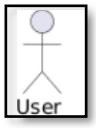
  - Return Message: Shows the response sent back to the sender.

  - Self-Message: A message where an object calls itself to perform another action.
  - Create / Destroy Message: Shows creation of a new object or termination of an object in the sequence.

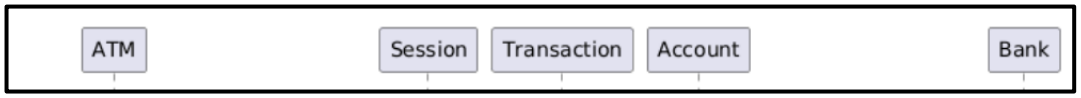


6(a)

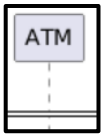
- **Aim:** Sequence Diagram of ATM System.
  The Sequence Diagram of the ATM system shows how different components interact with each other in a specific order to complete an ATM transaction. It represents the flow of messages between the User, ATM, Session, Transaction, Account, and Bank during authentication and transaction processing.
  The diagram mainly includes two phases: Authentication and Transaction Flow. It illustrates how the ATM verifies the user, creates a session, processes the selected transaction, communicates with the bank, and finally displays the result to the user.
  Participants:
  • User (Actor): Represents the customer who interacts with the ATM system.
  • ATM: The interface that receives user input and sends requests to other system components.
  • Session: Handles session creation and validation after authentication.
  • Transaction: Represents the transaction instance created when a user selects a transaction.
  • Account: Processes account-related operations such as withdrawal or deposit.
  • Bank: Confirms and updates the transaction in the banking system.
  Sequence of Interaction:
  • Insert Card and Enter PIN: User inserts the card and enters the PIN in the ATM.
  • Create Session: ATM sends createSession(sessionId) to Session to create a user session.
  • Authentication Status: Session verifies the user and returns authenticationStatus = true.
  • Select Transaction: User selects a transaction using selectTransaction(type, amount).
  • Validate Session: ATM sends validateSession() to ensure the session is valid.
  • Create Transaction: ATM creates a transaction instance using create(transactionId).
  • Process Transaction: ATM requests Account to process the transaction using processTransaction(type, amount).
  • Bank Update: Account sends requestUpdate(type, amount) to the Bank.
  • Transaction Confirmation: Bank confirms the transaction and sends transactionConfirmation.
  • Display Result: ATM displays the transaction result to the user using displayResult().
  Components:
  • Actor: Represents the external user interacting with the ATM system (User).



  • Participants (Objects): System components involved in the interaction (ATM, Session, Transaction, Account, Bank.



  • Lifeline: A vertical dashed line that represents the existence of a participant during the interaction.

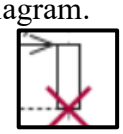


- Activation Bar: A rectangle on the lifeline showing when an object is performing an operation.

- Message: Arrows representing communication between participants (e.g., createSession(), processTransaction()).

- Return Message: Shows the response sent back after processing a request (e.g., authenticationStatus, transactionSuccess).

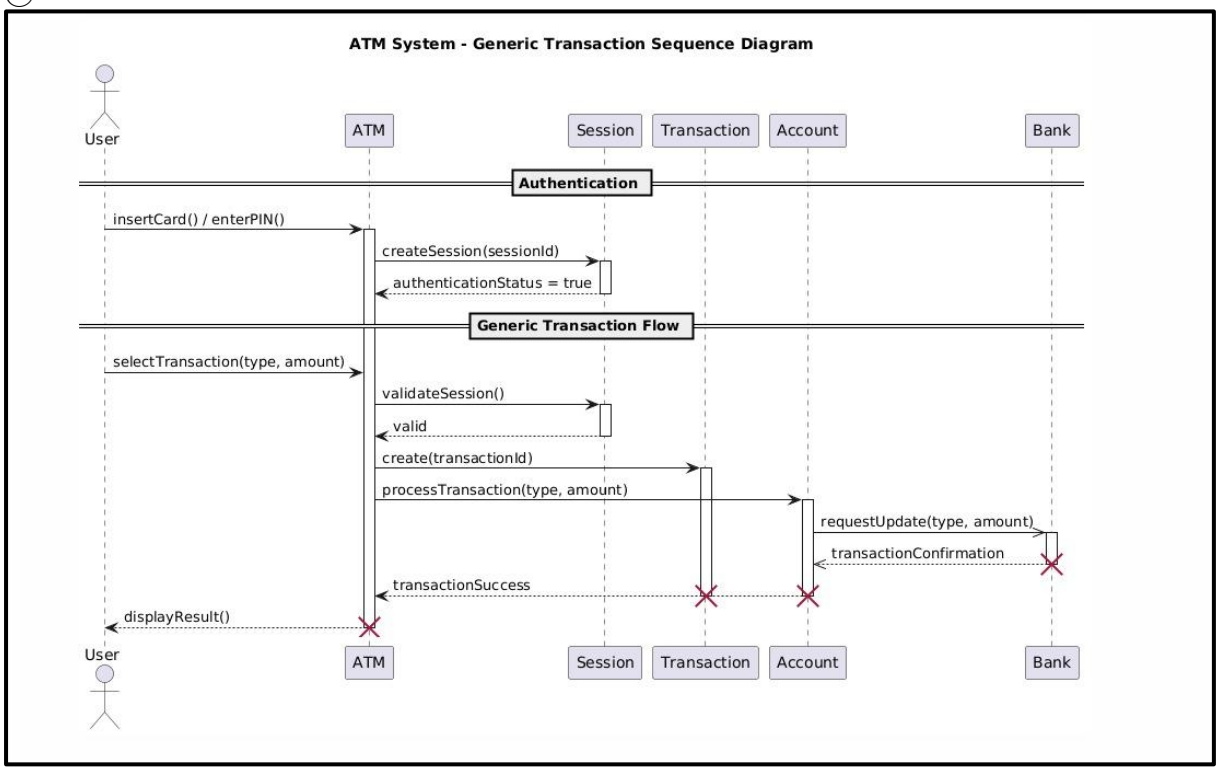
- Destroy Message: Indicates that a participant object is terminated. Shown by an "X" mark at the end of the lifeline in the diagram.



- **CODE:**

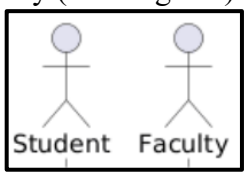
```
@startuml
title ATM System – Generic Transaction Sequence Diagram

actor User
participant ATM
participant Session
participant Transaction
participant Account
participant Bank

== Authentication ==

User -> ATM : insertCard() / enterPIN()
activate ATM

ATM -> Session : createSession(sessionId)
activate Session
Session --> ATM : authenticationStatus = true
deactivate Session

== Generic Transaction Flow ==
User -> ATM : selectTransaction(type, amount)
ATM -> Session : validateSession()
activate Session
Session --> ATM : valid
deactivate Session
```

```
ATM -> Transaction : create(transactionId)
activate Transaction

ATM -> Account : processTransaction(type, amount)
activate Account

Account ->> Bank : requestUpdate(type, amount)
activate Bank
Bank -->> Account : transactionConfirmation
destroy Bank
Account --> ATM : transactionSuccess
destroy Account
destroy Transaction
ATM --> User : displayResult()
destroy ATM

@enduml
```



6(b)

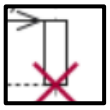
- **Aim:** Sequence Diagram of Amizone System.
  The Sequence Diagram of the Amizone system shows how different components interact with each other in a specific order when students or faculty members log in and use services in the Amizone portal. It represents the flow of messages between the Student, Faculty, Amity Server, Dashboard, Service, and Database during login authentication and service processing.
  The diagram mainly consists of two phases: Student Login and Service and Faculty Login and Service. It illustrates how the system validates user credentials, loads the dashboard, processes the selected service, communicates with the database, and finally displays the result to the user.
  Participants:
  • Student & Faculty (Actors): Users logging in and requesting services.
  • Amity Server: Processes login and service requests.
  • Dashboard: Loads the user interface after login.
  • Service: Handles specific student or faculty service requests.
  • Database: Validates users and provides data updates.
  Sequence of Interaction:
  • Users enter credentials → Server validates via Database → Returns validation result.
  • On success, Server loads Dashboard and processes requested service.
  • Service requests data update from Database and returns response.
  • Server displays results to users and handles logout.
  Components:
  • Actor: Represent Student and Faculty (stick figures).



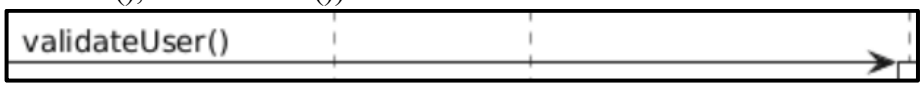
  • Participants (Objects): Represented as labeled rectangles (Server, Dashboard, Service, Database).

  • Lifeline: A vertical dashed line that represents the existence of a participant during the interaction.

  • Activation Bar: A rectangle on the lifeline showing when an object is performing an operation.

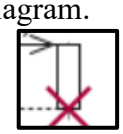


  • Message: Solid arrows between participants showing communication (e.g., enterCredentials(), validateUser()).



- Return Message: Shows the response sent back after processing a request (e.g., validationResult, dashboardLoaded, serviceCompleted).

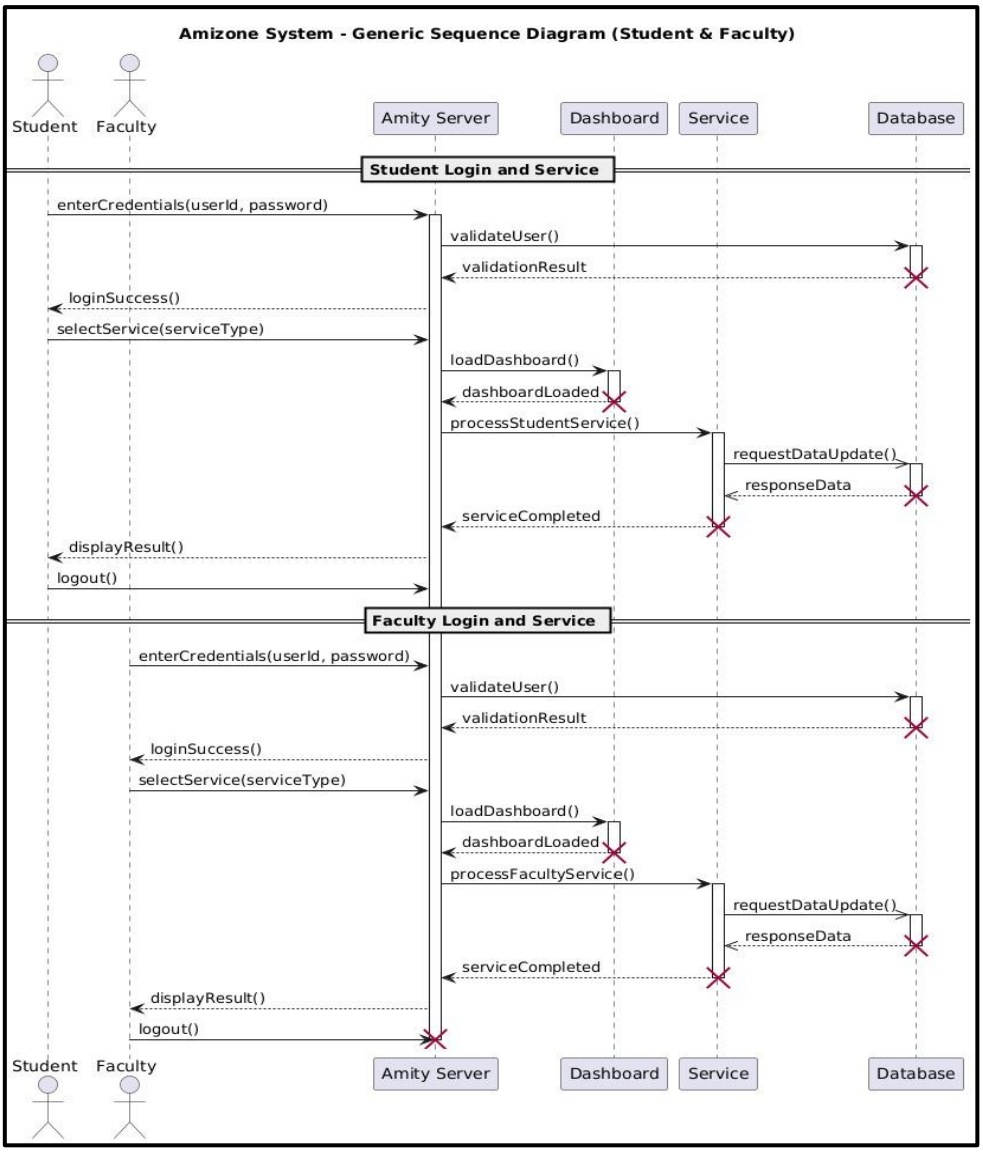
- Destroy Message: Indicates that a participant object is terminated. Shown by an "X" mark at the end of the lifeline in the diagram.



- **CODE:**

```
@startuml
title Amizone System – Generic Sequence Diagram (Student & Faculty)
actor Student
actor Faculty
participant "Amity Server" as Server
participant Dashboard
participant Service
participant Database
== Student Login and Service ==
Student -> Server : enterCredentials(userId, password)
activate Server
Server -> Database : validateUser()
activate Database
Database --> Server : validationResult
destroy Database
Server --> Student : loginSuccess()
Student -> Server : selectService(serviceType)
Server -> Dashboard : loadDashboard()
activate Dashboard
Dashboard --> Server : dashboardLoaded
destroy Dashboard
Server -> Service : processStudentService()
activate Service
Service ->> Database : requestDataUpdate()
activate Database
Database -->> Service : responseData
destroy Database
Service --> Server : serviceCompleted
destroy Service
Server --> Student : displayResult()
Student -> Server : logout()
== Faculty Login and Service ==
Faculty -> Server : enterCredentials(userId, password)
Server -> Database : validateUser()
activate Database
Database --> Server : validationResult
destroy Database
```

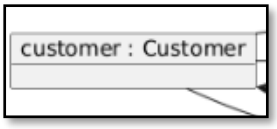
```
Server --> Faculty : loginSuccess()
Faculty -> Server : selectService(serviceType)
Server -> Dashboard : loadDashboard()
activate Dashboard
Dashboard --> Server : dashboardLoaded
destroy Dashboard
Server -> Service : processFacultyService()
activate Service
Service ->> Database : requestDataUpdate()
activate Database
Database -->> Service : responseData
destroy Database
Service --> Server : serviceCompleted
destroy Service
Server --> Faculty : displayResult()
Faculty -> Server : logout()
destroy Server
@enduml
```



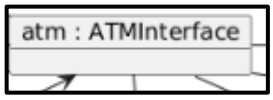
LAB Assignment 7

- **Theory:** A Collaboration Diagram (Communication Diagram) is a type of behavioral diagram in UML that shows how objects interact and communicate with each other to complete a task. It focuses on the relationships between objects and the messages exchanged between them.
  In this diagram, objects are placed freely, and numbered messages are used to show the order of communication between them. It helps in understanding how different parts of a system work together to perform a function.

  Components of Collaboration Diagram:
  - Actor: Represents a user or external entity that interacts with the system. It is usually shown as a stick figure and initiates the interaction with the system.



  - Object (Participant): Represents an instance of a class that participates in the interaction. It is displayed as a rectangle containing the object name and sometimes the class name.



  - Sequence Number: Numbers added before messages to show the order in which the messages are sent (for example: 1, 1.1, 2, 2.1).

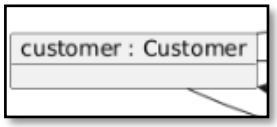
  - Link: A connection between two objects that shows they can communicate with each other. It represents the relationship through which messages are passed.

  - Message: Represents communication between objects. It is shown as an arrow along the link with a number indicating the order of the message in the interaction.

  - Self Message: A message that an object sends to itself to perform an internal operation.

7(a)

- **Aim:** Collaboration Diagram of ATM System.
  The Collaboration Diagram (Communication diagram) of the ATM system shows how different objects interact with each other to complete an ATM transaction. It represents the communication and relationships between system components and the numbered messages exchanged to perform operations.
  The diagram illustrates how the Customer, ATM Interface, Bank Server, Account Database, Cash Dispenser, and Receipt Printer communicate during an ATM withdrawal process. It shows how the user inserts the card, enters the PIN, selects a transaction, and how the ATM coordinates with the bank system to verify the account, process the transaction, dispense cash, and print the receipt.
  Components:
  - Actor: The external user interacting with the system (customer).



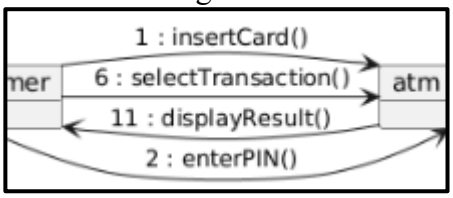
  - Object (Participant): The external user interacting with the system (atm : ATMInterface bankServer : BankServer).

  - Link:A connection between objects that shows they can communicate (Customer ↔ ATM Interface ATM Interface ↔ Bank Server).

  - Message: Arrows showing communication between objects (insertCard(), enterPIN()).

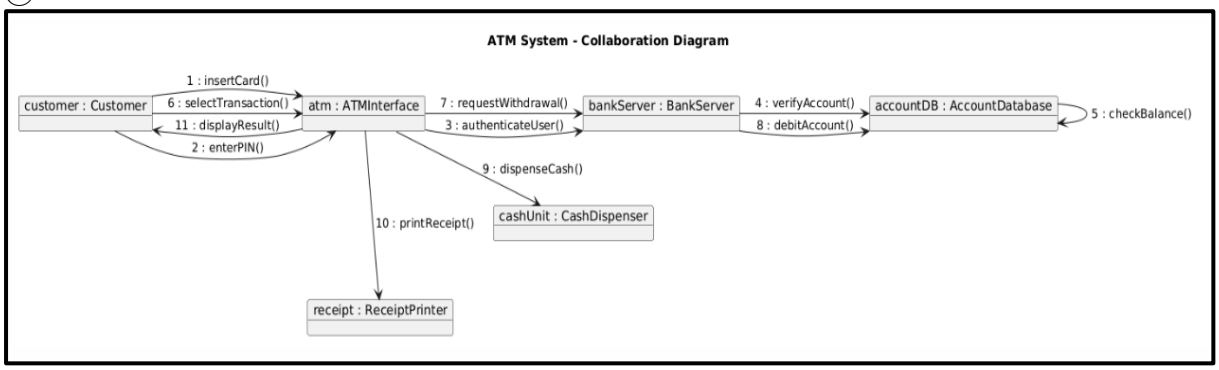
  - Self Message: When an object sends a message to itself.(e.gaccountDB: AccountDatabase → checkBalance().

  - Sequence Number: Sequence numbers indicate the order in which messages are exchanged between objects in the ATM collaboration diagram.



- **CODE:**
  @startuml

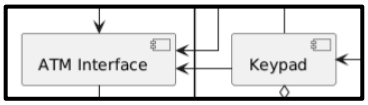
```
title ATM System - Collaboration Diagram

object "customer : Customer" as C
object "atm : ATMInterface" as ATM
object "bankServer : BankServer" as BS
object "accountDB : AccountDatabase" as DB
object "cashUnit : CashDispenser" as CD
object "receipt : ReceiptPrinter" as RP

' Layout structure
C -[hidden]down-> ATM
ATM -[hidden]down-> BS
BS -[hidden]-> DB
DB -[hidden]-> CD
CD -[hidden]-> RP

' Messages
C -> ATM : 1 : insertCard()
C -> ATM : 2 : enterPIN()

ATM -> BS : 3 : authenticateUser()

BS -> DB : 4 : verifyAccount()
DB -> DB : 5 : checkBalance()

C -> ATM : 6 : selectTransaction()
ATM -> BS : 7 : requestWithdrawal()

BS -> DB : 8 : debitAccount()

ATM -> CD : 9 : dispenseCash()
ATM -> RP : 10 : printReceipt()

ATM -> C : 11 : displayResult()

@enduml
```



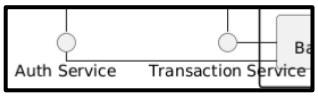
LAB Assignment 8

- **Theory:** A Component Diagram is a type of structural diagram in UML that shows how different components of a system are organized and interact with each other. It represents the physical structure of the system, including software modules and subsystems.
  Components are shown as rectangles, and their connections or dependencies are represented using connectors or arrows. This diagram helps in understanding the high-level architecture of a system and how its components work together.
  Components of Component Diagram:
  - Actor: Represents the user or external entity interacting with the system.

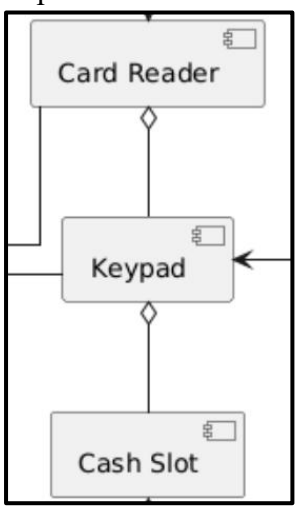
  - Component: Modular part of the system that provides specific functionality (e.g., ATM Interface, Authentication Module).



  - Interface: Defines the services a component provides or requires to communicate with others.



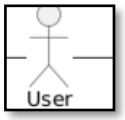
  - Dependency: Shows that one component relies on another to function.



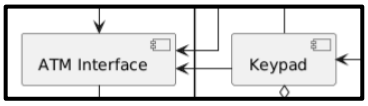
  - Port: Interaction point through which a component communicates with other components or external systems.

8(a)

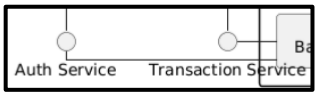
- **Aim:** Component Diagram of ATM System.
  The Component Diagram of the ATM System represents the high-level architecture of the ATM application. It shows how different software components are organized and how they interact to perform ATM operations.The diagram illustrates components such as ATM Interface, Authentication Module, Transaction Processor, Bank Server, Account Database, Cash Dispenser, and Receipt Printer.
  It explains how the customer interacts with the ATM interface, which communicates with other components to authenticate the user, process the transaction, update account information, dispense cash, and print the receipt. This diagram helps in understanding the modular structure of the ATM system and how each component contributes to completing a transaction..
  Components:
  - Actor: The external user interacting with the system (user).



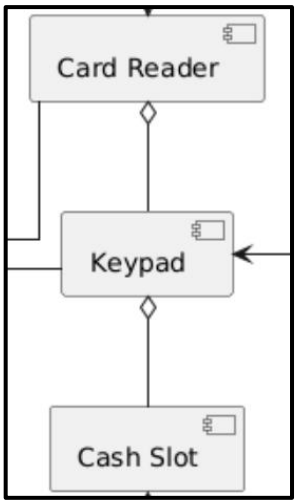
  - Component: Modular parts of the system representing physical or logical units (such as: ATM Machine,Card Reader).



  - Interface: Defined sets of services components provide or require to interact, such as Card Service, PIN Service, Transaction Service, Account Service, Cash Service, and Print Service.



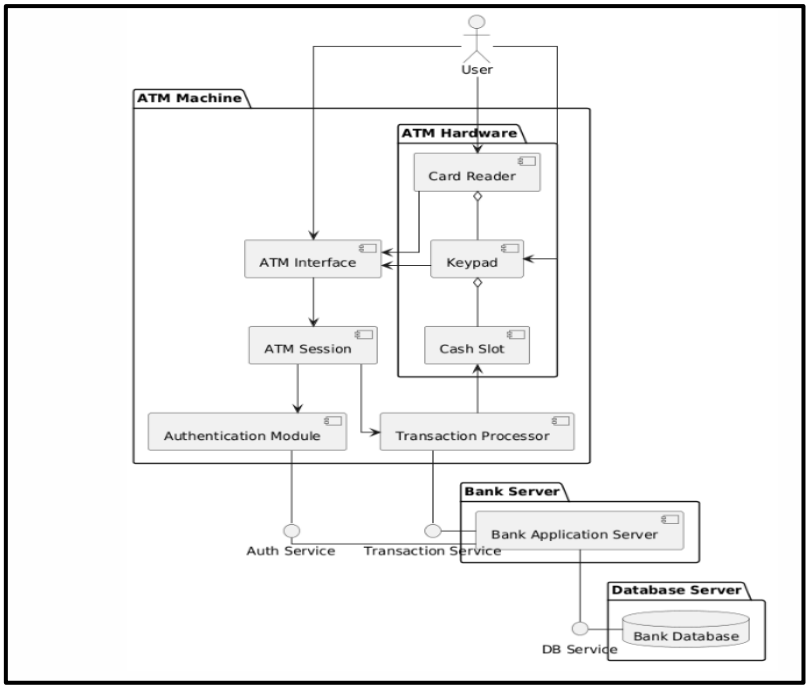
  - Dependency: A relationship showing that one component relies on another to fulfill its function (e.g., ATM Machine depends on Card Reader for card reading).



- **CODE:**

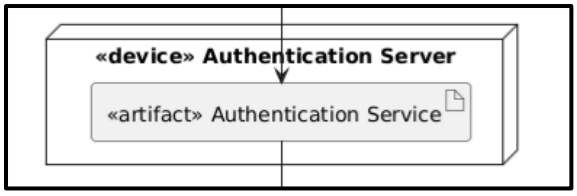
```
@startuml
skinparam componentStyle uml2
skinparam linetype ortho
actor User
```

```
package "ATM Machine" {
  component "ATM Interface" as UI
  component "ATM Session" as Session
  component "Authentication Module" as Auth
  component "Transaction Processor" as TP
  package "ATM Hardware" {
      component "Card Reader" as CR
      component "Keypad" as KP
      component "Cash Slot" as CS
      CR o-- KP
      KP o-- CS  }}
package "Bank Server" {  component "Bank Application Server" as BAS}
package "Database Server" {  database "Bank Database" as DB}
' Interfaces
interface "Auth Service" as IAuth
interface "Transaction Service" as ITrans
interface "DB Service" as IDB
' Associations
User --> UI
User --> CR
User --> KP
CR --> UI
KP --> UI
TP --> CS
UI --> Session
Session --> Auth
Session --> TP
' Dependencies via interfaces
Auth -- IAuth
IAuth - BAS
TP -- ITrans
ITrans - BAS
BAS -- IDB
IDB - DB
@enduml
```



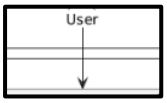
LAB Assignment 9

- **Theory:** A Deployment Diagram is a type of structural diagram in UML that shows the physical deployment of artifacts (software components, executables) on nodes (hardware or virtual devices) in a system. It focuses on how the software is distributed across hardware and how the nodes interact to execute the system.
  Deployment diagrams are particularly useful for understanding system architecture, hardware requirements, and communication paths between different nodes. They are often used in the design and planning of system installations and networked applications.
  Components of Deployment Diagram:
  - Node: Represents a physical device or execution environment in the system, such as a server, workstation, or mobile device. It is depicted as a 3D box or cube.



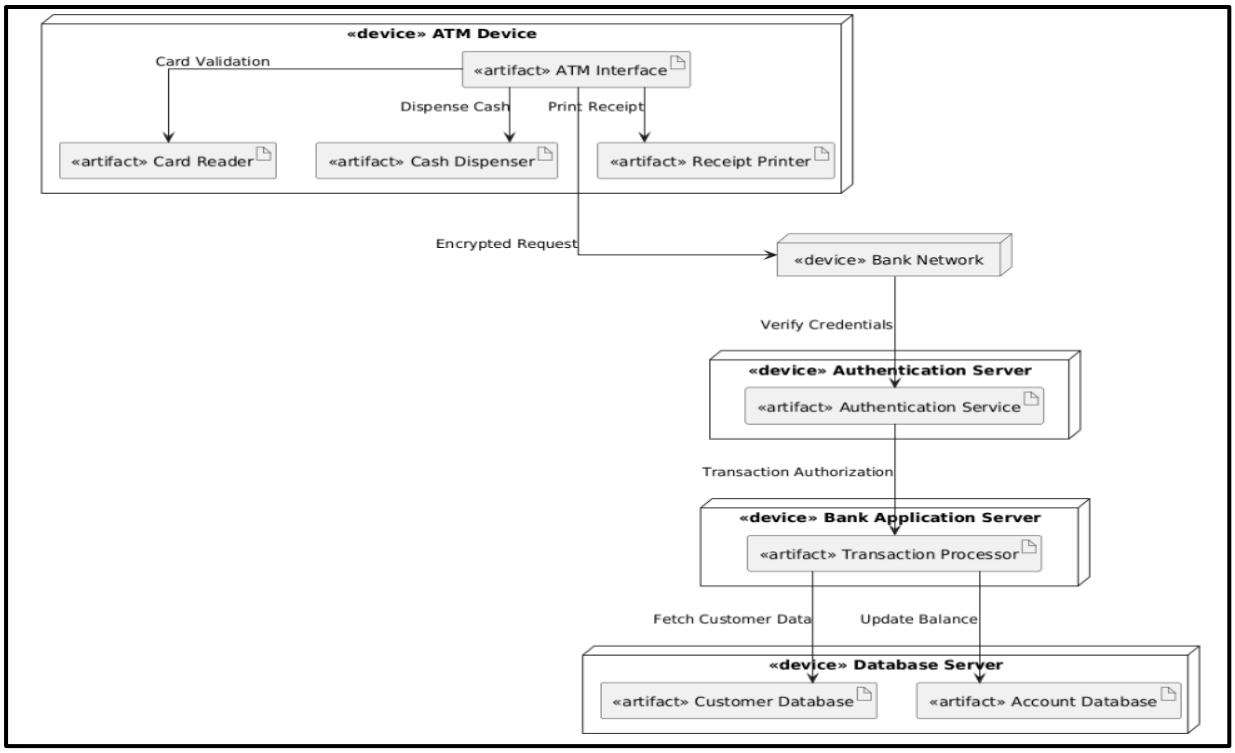
  - Artifact: Represents a physical piece of software that is deployed on a node, such as an executable file, library, or database. Artifacts are shown as rectangles with a small folded corner.

  - Communication Path (Link): Represents the connection between nodes, indicating how nodes communicate with each other. It is shown as a line connecting the nodes.



  - Dependency: Shows that an artifact or node depends on another artifact or node for its execution.
  - Multiplicity/Constraints: Shows how many instances of a node or artifact exist or any specific conditions for deployment.

9(a)

- **Aim:** Deployment Diagram of ATM System.
  The Deployment Diagram of the ATM System illustrates the physical architecture showing how software artifacts are deployed on hardware nodes. It demonstrates how the ATM system components are distributed across physical devices such as the ATM Machine, Bank Server, and Bank Database Server. The diagram visualizes the real-world arrangement of hardware and software components and their communication paths to support ATM operations.
  This deployment view helps in understanding the physical setup of the system, including where software runs, how devices connect, and how data flows between these physical nodes during a transaction.
  Analysis of the diagram:
  - ATM Device Node:
    - Represents the physical ATM terminal that customers use to perform banking activities.
    - It includes several artifacts such as the ATM Interface, Card Reader, Cash Dispenser, and Receipt Printer, which manage user interaction and carry out transactions.
  - ATM Interface Artifact
    - Responsible for controlling the ATM's overall functionality and coordinating communication with hardware components like the Card Reader, Cash Dispenser, and Receipt Printer.
  - Card Reader Artifact: Detects and reads the customer's ATM card and verifies the card information before allowing the user to continue with any operation.
  - Cash Dispenser Artifact: Handles the release of cash to the user when a withdrawal transaction is successfully completed.
  - Receipt Printer Artifact: Generates and prints a receipt that contains the details of the transaction performed by the user.
  - Bank Network Node: Serves as the communication medium that connects the ATM terminal to the bank's backend servers.
  - Authentication Server Node: Hosts the Authentication Service artifact, which checks and validates user credentials such as the PIN.
  - Bank Application Server Node: Contains the Transaction Processor artifact, which manages and processes the banking operations requested by the ATM machine.
  - Database Server Node: Maintains banking information using artifacts like the Customer Database and Account Database.
  - Communication Process: The ATM terminal sends a secure request through the bank network.
    - This request is directed to the Authentication Service to verify the user's credentials.
    - Once authentication is successful, the Transaction Processor handles the requested operation.
    - The processor then interacts with the Customer and Account Databases to obtain the necessary information and update account balances accordingly.
- **CODE:**
  @startuml
  skinparam linetype ortho

```
node "<<device>> ATM Device" {
   artifact "<<artifact>> ATM Interface"
   artifact "<<artifact>> Card Reader"
   artifact "<<artifact>> Cash Dispenser"
   artifact "<<artifact>> Receipt Printer"
   "<<artifact>> ATM Interface" --> "<<artifact>> Card Reader" : Card Validation
   "<<artifact>> ATM Interface" --> "<<artifact>> Cash Dispenser" : Dispense Cash
   "<<artifact>> ATM Interface" --> "<<artifact>> Receipt Printer" : Print Receipt
}
node "<<device>> Bank Network"
node "<<device>> Authentication Server" {
   artifact "<<artifact>> Authentication Service"
}
node "<<device>> Bank Application Server" {
   artifact "<<artifact>> Transaction Processor"
}
node "<<device>> Database Server" {
   artifact "<<artifact>> Customer Database"
   artifact "<<artifact>> Account Database"
}
"<<artifact>> ATM Interface"---> "<<device>> Bank Network" : Encrypted Request
"<<device>> Bank Network"---> "<<artifact>> Authentication Service" : Verify Credentials
"<<artifact>> Authentication Service"---> "<<artifact>> Transaction Processor" : Transaction
Authorization
"<<artifact>> Transaction Processor"---> "<<artifact>> Customer Database" : Fetch Customer Data
"<<artifact>> Transaction Processor"---> "<<artifact>> Account Database" : Update Balance
@enduml
```